



Information Systems Department
Faculty of Computers and Artificial Intelligence
Cairo University



Problem Solving Assistant

PSA

Under Supervision of:

Dr. Ali Zidan

TA. Nourhan Ihab

Prepared By

Marwan Ahmed Abd El-Monem

20210378

Farah Mohammed Hussien

20210287

Eslam Sayed Gouda

20211012

Bassem Wael Mohammed

20210085

Ahmed Khaled Erfan

20210015

Ali Tarek Mohammed

20210245

Academic Year 2024/2025

Final year Document

July 9th, 2025

Table of Content

1	Chapter (1) Introduction	6
1.1	Problem Statement	6
1.2	Objectives	7
1.3	Solution.....	7
1.4	Scope.....	7
1.5	Methodology	8
1.5.1	Module 1: From Problem to Approach and from Requirements to Collection.....	9
1.5.2	Module 2: From Understanding to Preparation and from Modelling to Evaluation	10
1.5.3	Module 3: From Deployment to Feedback	10
2	Chapter (2) Market Analysis	11
2.1	Requirement Validation	11
3	Chapter (3) Analysis	13
3.1	Project stakeholders	13
3.2	Functional requirements.....	13
3.3	Non-functional requirements	14
3.4	User Profiles	14
3.5	Personas	15
3.6	Use Case Diagram.....	16
3.7	Class Diagram.....	17
3.8	Entity Relationship Diagram.....	18
3.9	MongoDB Database Schema	18
3.9.1	Problems Collection Schema	19
3.9.2	Submissions Collection Schema	19
3.10	Sequence Diagrams.....	20
3.10.1	Debug Code Sequence Diagram	20
3.10.2	Request Roadmap Sequence Diagram	21
3.10.3	Request Solution Sequence Diagram	21
3.11	Use Case Narratives.....	22
3.12	Gantt chart.....	32
4	Chapter (4) PSA Design	33
4.1	UI Design	33
4.1.1	Mobile UI Design.....	33
4.1.2	Web UI Design.....	34
4.2	Implementation	36
4.2.1	Asking for a roadmap.....	36

4.2.2	Ask for a solution.....	37
4.2.3	Ask for a Code debug.....	38
4.2.4	Online Judge Insights Integration	40
4.3	Used Tools	43
4.4	Courses.....	54
4.4.1	IBM Data Science Professional Certificate.....	54
4.4.2	Flutter Course.....	54
4.4.3	Django Course	54
5	Chapter (5) Testing and Evaluation	55
5.1	Testing Objectives	55
5.2	Testing Types and Tools.....	55
5.3	Backend Test cases	55
5.3.1	Authentication Test cases.....	55
5.3.2	User Management Test cases	76
5.3.3	Chat Test cases.....	83
5.3.4	Profile Management Test cases.....	87
5.4	Front-End Mobile Test Cases	90
5.4.1	Chat Test cases.....	90
5.4.2	User Profile Management Test cases	93
5.4.3	Authentication Test cases.....	102
5.5	Front-end Web Test cases.....	125
5.5.1	Chat Test cases.....	125
5.5.2	User Profile Management Test cases	129
5.5.3		137
5.5.4	Authentication Test cases.....	138
6	Chapter (6) Conclusions and Future Work	162
6.1	Conclusions.....	162
6.2	Future Work.....	162
7	References	163

List of Figures

Figure 1-1:Data Science Methodology (IBM)	8
Figure 2-1: Most Sites People Solve On.....	11
Figure 2-2:Average Time Taken To Reach Each Rank.....	12
Figure 2-3: What do you feel like you are missing?	12
Figure 3-1: Use case diagram	16
Figure 3-2:Class Diagram	17
Figure 3-3:Entity Relationship Diagram.....	18
Figure 3-4:NOSql MongoDB Schema	18
Figure 3-5:Debug Code Sequence Diagram	20
Figure 3-6:Request Roadmap Sequence Diagram	21
Figure 3-7:Request Solution Sequence Diagram	21
Figure 3-8: Project TimeLine.....	32
Figure 4-1: Mobile UI Design.....	33
Figure 4-2: Web Home Screen, Register, Login, and Check Email	34
Figure 4-3:Web Reset Password, Chat Pages, setting	34
Figure 4-4: Web User Profile, and User statistics.....	35
Figure 4-5: Web Session Expired	35
Figure 4-6: Asking for consequent topic	36
Figure 4-7: Asking for a problem solution	37
Figure 4-8:Chat response	37
Figure 4-9 Rest of response	38
Figure 4-10 simple code needs to be debugged.....	39
Figure 4-11Response with debugged code	39
Figure 4-12 Rest of debugged code response	40
Figure 4-13 Codeforces rates	41
Figure 4-14 Codeforces topics	41
Figure 4-15 Atcoder insights	42

List of Tables

Table 3-1: User Profile.....	14
Table 3-2: Register Use Case Table.....	22
Table 3-3:Login Use case Table	24
Table 3-4: Request RoadMap Use Case Table	26
Table 3-5: Request a solution (using Text) use case Table	27
Table 3-6: Convert algorithm to code Use case table	28
Table 3-7:Request Code Debug And/ Or Correction use case table.....	30
Table 4-1:Python Usage.....	43
Table 4-2: : Flutter Usage	44
Table 4-3 : DrawIo Usage.....	45
Table 4-4: MongoDB Usage	45
Table 4-5: PostgreSQL Usage	46
Table 4-6: Figma Usage.....	46
Table 4-7:GitHub Usage	47
Table 4-8: Google Drive Usage	48
Table 4-9 : Zoom Usage	49
Table 4-10: Microsoft PoweBI usage	50
Table 4-11: Django Usage	50
Table 4-12: Pycharm Professional Usage	51
Table 4-13: React Usage	51
Table 4-14: Hugging Face Usage	52
Table 4-15: Selenium Usage	52
Table 4-16: AnyDesk Usage.....	53
Table 4-17: DeepSeek Usage.....	53

1 Chapter (1) Introduction

1.1 Problem Statement

Problem solving is an essential skill for every computer scientist, it changes the mind-set of the person by strengthening their ability to think out of the box when solving any type of problem, not only in the code development part, but in real life as well.

Problem-Solving (Real-Life Skill) vs. Problem-Solving (Coding):

In everyday life, problem-solving involves abstract thinking, creativity, and handling ambiguous situations. It applies to various contexts, from personal decision-making to complex business scenarios. Coding problem-solving, however, is more structured. The problems are usually well-defined, requiring logical reasoning, algorithmic thinking, and technical expertise. Essentially, coding problem-solving is a focused extension of general problem-solving skills, but it's bound by the specific constraints of programming languages, system design, and computational efficiency. In this project, we focus on coding problem-solving, as it forms the foundation for technical problem-solving in the field of computer science.

For computer scientists, mastering problem-solving can open doors to prestigious companies, as these companies prioritise critical thinking in general, which is a skill that problem solving enriches.

Challenges in Developing Problem-Solving Skills:

Given the importance of such skill, many computer scientists strive to develop said skill, but they face many challenges, such as not knowing how to start, getting stuck in a problem and not finding a reliable solution, not knowing who to approach the problem, not knowing what tool to use to optimize memory, time, and cost.

Also being a skill which requires strategic thinking, many decide to compete in competitions, in which they solve a problem with certain constraints in the quickest way possible which improves not only the strategy to approach a problem but also the speed of solving a problem. But when participating in competitive programming competitions, many face challenges such as not knowing what the solver's current state is, what to do next which may cause them stress and navigate them into getting private coaches which not a lot of people can afford.

1.2 Objectives

Our objective is to give computer science students, competitive programmers, and everyone seeking knowledge and experience in competitive programming & problem solving ways to approach a problem with reliable solutions to problems they want to solve considering the time, memory and cost constraints.

As well as evaluate competitive programmers, in addition to providing them with an overview to their current level of skill and how to progress to improve their skills leads them to enhance their skills and advancing to higher positions/ranks in the competitions.

1.3 Solution

After observing the struggles that competitive programmers and novice practitioners face when solving problems, and analysing data from problem-solving sites, we decided to develop a ChatBot to address these challenges. Our ChatBot offers an interactive, flexible solution tailored to the user's needs.

This ChatBot takes the problem that the person is facing and provides the best way to approach such a problem and the code for it (if asked) while optimizing the code relative to the constraints provided in the statement or choosing the general optimal solution based on time, memory and cost if no other constraints are provided.

The ChatBot also will be able to receive solutions to a problem from the user, analysing those solutions and what they are solving and what is wrong with them on a syntax level and logical level.

Also the ChatBot would have the ability to improve a competitive programmer's positions/ ranks by taking their current rank (possibly on problem-solving sites) and/or the current knowledge of topics, and suggests problems/ topics to solve, as well as giving them a roadmap on what to do next, such as ways to approach a topic, how to improve the speed of solving a problem, all without the need of interference from any private coaching.

1.4 Scope

This program will provide a solution to the problem using an algorithm and/or its C++ interpretation, it will not include any other programming language as C++ is the most practiced language when it comes to problem-solving, and more languages will be added such as python and java in the future.

It will also help competitive programmers reach a certain level (Top 50 in national contests) in great accuracy and will not be focused on levels higher than that yet, as it will be done in the future.

1.5.1 Module 1: From Problem to Approach and from Requirements to Collection

Business Understanding:

What is the problem that you are trying to solve?

1. Establishing a **clearly defined question** starts with understanding the **GOAL** of the person who is asking the question.
2. Once the goal is clarified, the next piece of the puzzle is **to figure out the objectives that are in support of the goal.**

Analytics Approach:

How can you use the data to answer the question?

To determine which analytical approach to use, ask relevant questions about the problem. Depending on which question aligns best with your situation, you can choose the appropriate analytical method.

- 1) **Descriptive Questions:** “What is the current status?”
- 2) **Diagnostic Questions:** “Why did it happen?”
- 3) **Predictive Questions:** “What is likely to happen?”
- 4) **Prescriptive Questions:** “What should we do?”
- 5) **Classification Questions:** “Which category does this belong to?”

Data Requirements:

What are the data requirements?

1. Which data is required?
2. How to source or collect the data?
3. How to understand or work with the data?
4. And how to prepare the data to meet the desired outcome?

Data Collection

In this phase the data requirements are revised and decisions are made as to whether or not the collection requires more or less data.

1.5.2 Module 2: From Understanding to Preparation and from Modelling to Evaluation

Data understanding:

Is the data that you collected representative of the problem to be solved?

Data Preparation

To work effectively with the data, it must be prepared in a way that addresses missing or invalid values and removes duplicates, toward ensuring that everything is properly formatted. **Feature engineering** is also part of data preparation.

Modelling

- First, understand the question at hand.
- Second, select an analytic approach or method to solve the problem.
- Third, obtain, understand, prepare, and model the data.

Evaluation

Does the model used answer the initial question or does it need to be adjusted?

1.5.3 Module 3: From Deployment to Feedback

Deployment

Once the model is evaluated and the data scientist is confident it will work, it is deployed and put to the ultimate test.

Feedback

Once in play, feedback from the users will help to refine the model and assess it for performance and impact.

2 Chapter (2) Market Analysis

After understanding and defining the problem statement and grasping the basic requirements for the system. In this chapter, we will cover all the requirements in detail by identifying our main users and understanding what they want and need. So, we started by collecting data on the users using surveys.

2.1 Requirement Validation

We have found that using Google Forms for the survey is the best choice for gathering data because our target audience uses the technology a lot, which is the fastest and easiest way to collect data.

We began our survey with “What site do you mostly solve on” to know what is the main concern of the interviewees Competitive programming or Problem-solving for work, which is shown in **figure 2-1**

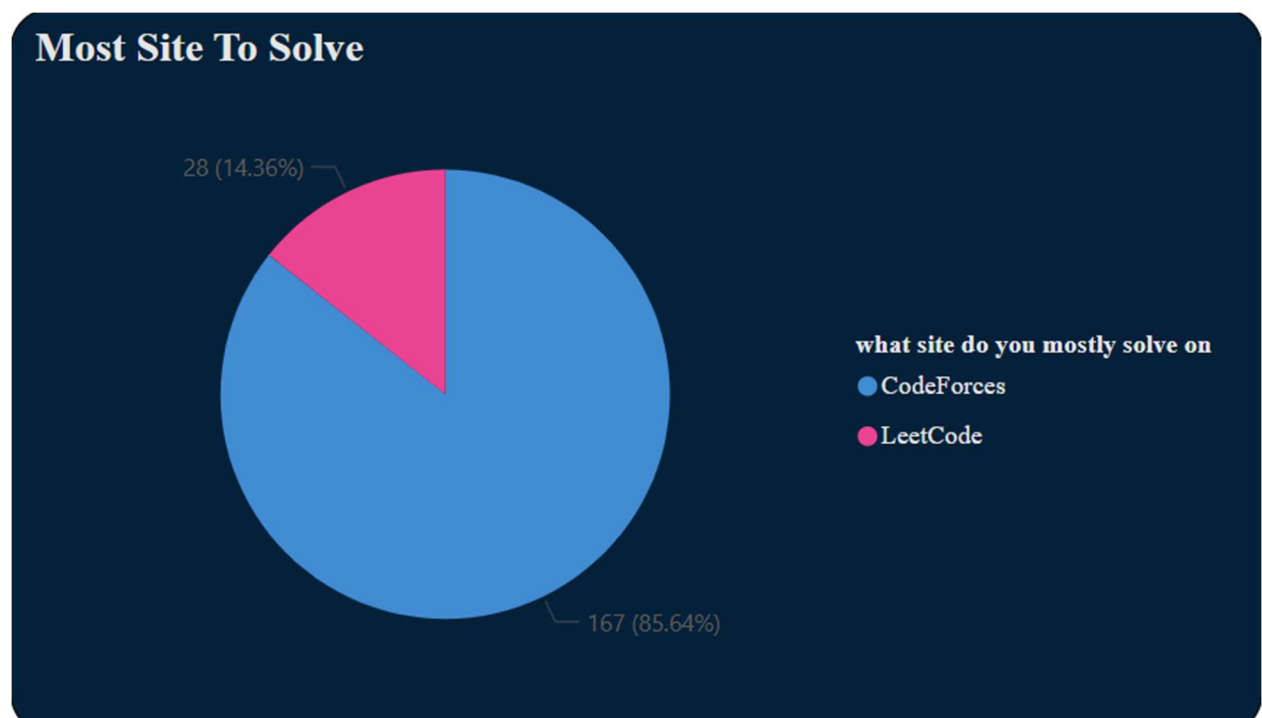


Figure 2-1: Most Sites People Solve On

We have found that the number of users interested in competitive programming is *Larger* than the number of users who need it for preparing for interviews or Similar things CP websites

And for the following questions the common questions in **Figure 2-2**

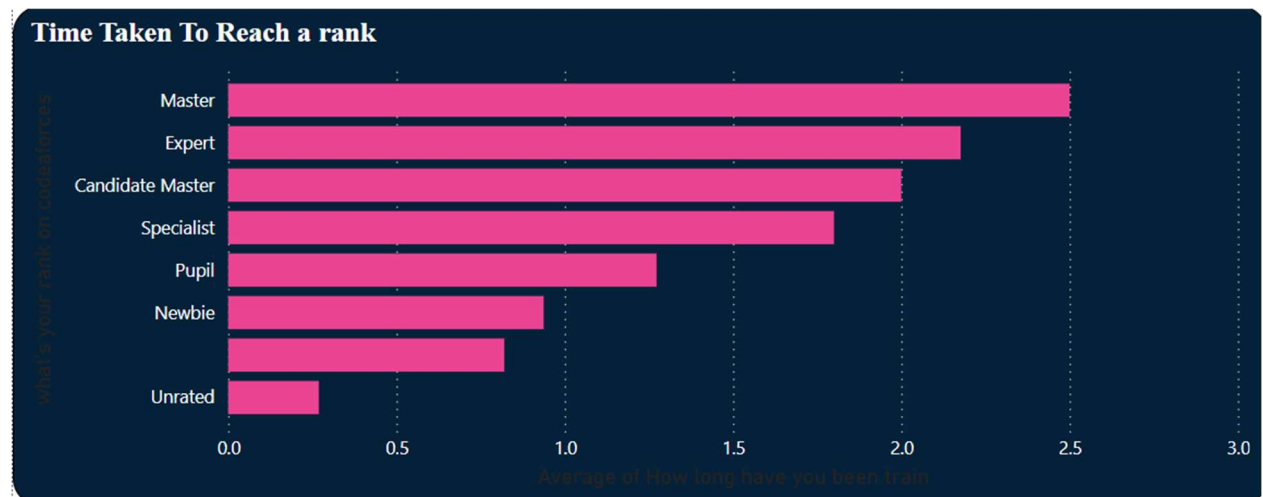


Figure 2-2: Average Time Taken To Reach Each Rank

In **Figure 2-3** we asked the people in the questioner: what do you think is missing within yourself in order to develop your problem-solving skills?

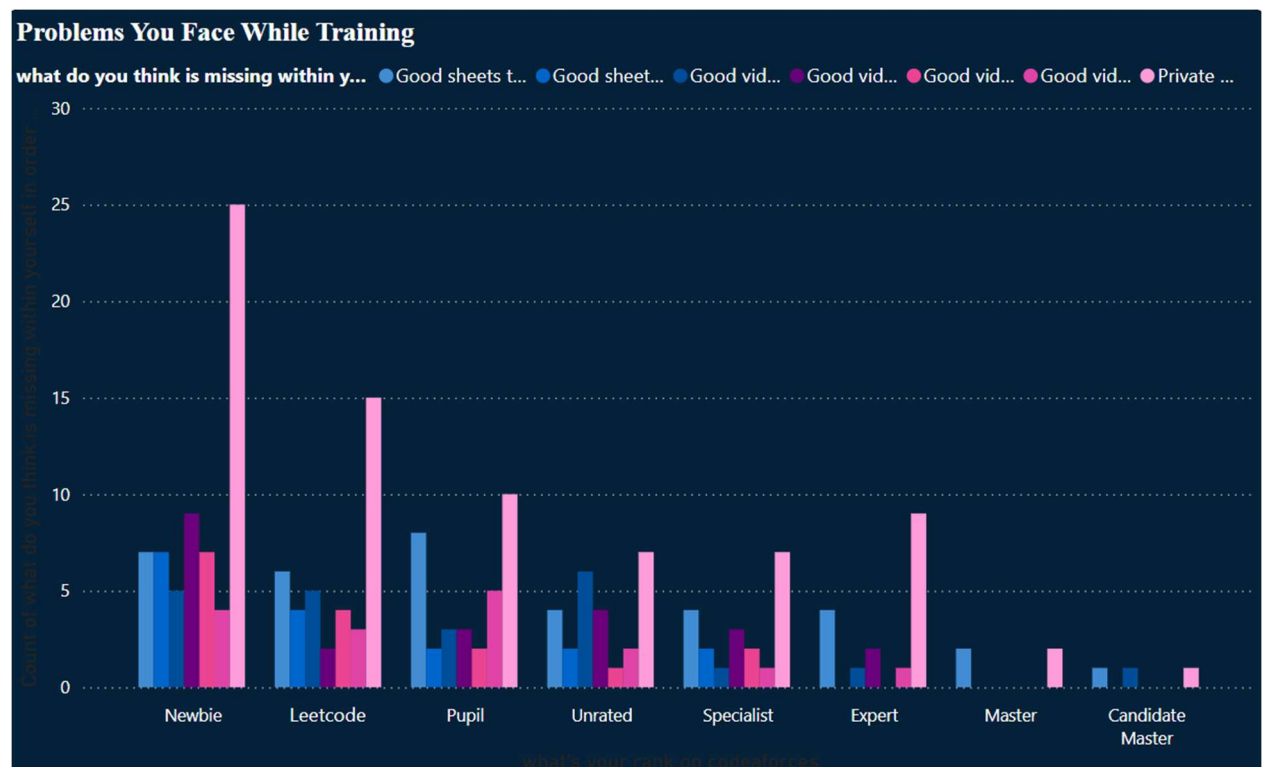


Figure 2-3: What do you feel like you are missing?

3 Chapter (3) Analysis

3.1 Project stakeholders

Stakeholders are individuals or groups that have an interest in any decisions or activities of the project. In the system, there are 3 stakeholders.

- First is the target user, as mentioned above, computer science students and competitive programmers who wish to develop their problem-solving skills.
- Second, any person who works on developing the system from the business Understanding phase to the deployment and maintenance phases (Project Team Members). Also, someone who supervised and guided the developers through the project (Academic Staff Supervisors).
- Third, any third-party suppliers

3.2 Functional requirements

- User can create an account by entering his/her userName, email, Password, and optionally their username/email on problem-solving accounts or by Google account / GitHub account
- Add or change problem-solving website credentials.
- User can login to his account by the saved credentials or by Google account / GitHub Account
- Users can ask about a roadmap for developing his/her skills based on their problem-solving accounts and some additional info such as the topics he/she knows until now. User can enter a problem statement using text to get a reliable solution for said problem.
- Users can enter a problem statement with constraints (time, memory) to get the best possible solution.
- Users can enter steps or an algorithm for a problem, to get the code for said Algorithm.
- Users can enter a code for the ChatBot to debug and correct the mistakes found.

Users can see Statistics on the problem solving website.

3.3 Non-functional requirements

- Usability: We target that the system will be easy to use and simple. Therefore, we will ensure to provide a system, you can easily navigate, understand, and remember by:
 - Following the common type of the chat window
 - Applying a tutorial for the other features except the chatting
 - Tell the user all the features he could use
 - Building the application with easy navigation to ensure that every task will be clear and accessible.
- Response Time: Try to ensure that the response time does not exceed 2:00 minutes considering the time of model reasoning and response time (which can be neglected due to its small time) by using good servers
- Operating System: Provide the system on multiple mobile OSs as in cross platform application and on a website
- Security Requirements: User passwords should be encrypted using SHA-256. The system should withstand 1000 login attempts per second (DDoS attack).

3.4 User Profiles

Table 3-1: User Profile

Characteristic	Decision Makers
Age	9 and above
Language	English
Education	Have a good knowledge with English and math
Computer experience	Low to Medium
Domain expertise	Medium to High
Task knowledge	Initially High
Expectations	Ease of use, helpful, speed of task, does the job needed, simple UI

3.5 Personas

Mohammed– The Newcomer with Curiosity but No Guidance

Ahmed is an 18-year-old first-year Computer Science student at Cairo University. Although he's just starting his academic journey, competitive programming feels like an intimidating world to him. He has no prior experience with coding or algorithms and doesn't know many people in the field to guide him. Ahmed is eager to learn, but without proper support or direction, he feels lost and overwhelmed by where to begin. He needs a clear, beginner-friendly path that introduces him to competitive programming without assuming any prior knowledge.

Ali – The Coder Who Hates Searching

Ali is also 18 and in his first year at university. Unlike Ahmed, he has some experience with programming and can write code confidently. However, he has never tried competitive programming before. His biggest frustration is searching for resources online—he finds it time-consuming, confusing, and often unhelpful. What Ali needs is a structured, straightforward learning experience that helps him dive into competitive programming with as little online searching and guesswork as possible.

Osama – The Talented Teen Looking for a Mentor

At just 16, Osama is a high school student who already has a solid grasp of competitive programming fundamentals. He's motivated and talented, but he's reaching the point where self-learning isn't enough. He doesn't just want to solve problems—he wants to grow, refine his skills, and identify his weak spots. What Osama needs is a flexible mentor or assistant who can guide him personally, help him debug code, and suggest what to learn next based on his current level.

Lily – The Ambitious Competitor Seeking Redemption

Lily is a 20-year-old university student in her third year. She's already stepped into the world of competitive programming and has even participated in two CPC contests—but both attempts ended in disappointment. Her dream is to earn a top rank in a major competition, but she's losing confidence. What Lily truly needs is someone who believes in her potential, helps her analyze her past mistakes, and gives her a clear plan to finally achieve the success she's been striving for.

3.6 Use Case Diagram

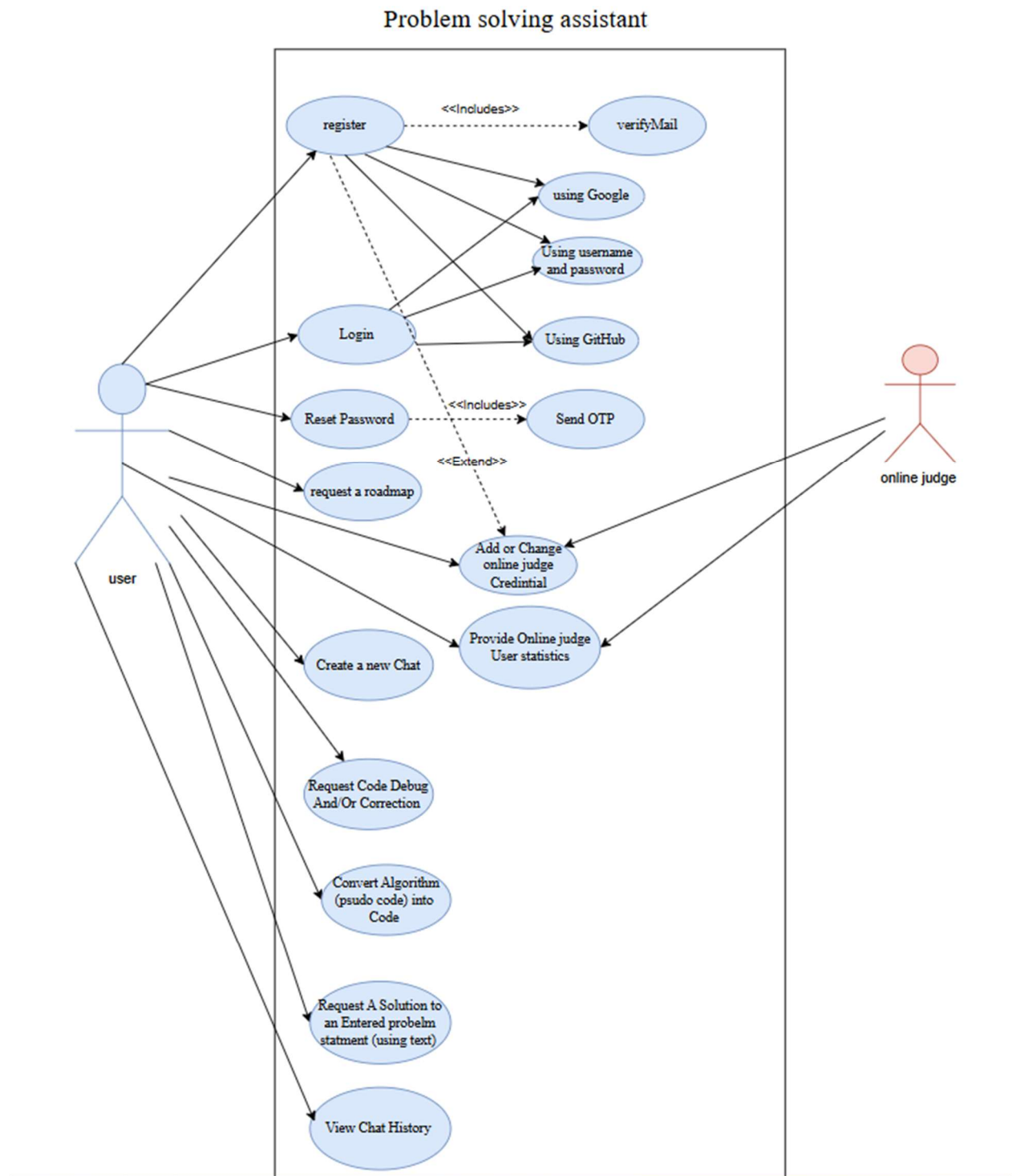


Figure 3-1: Use case diagram

3.7 Class Diagram

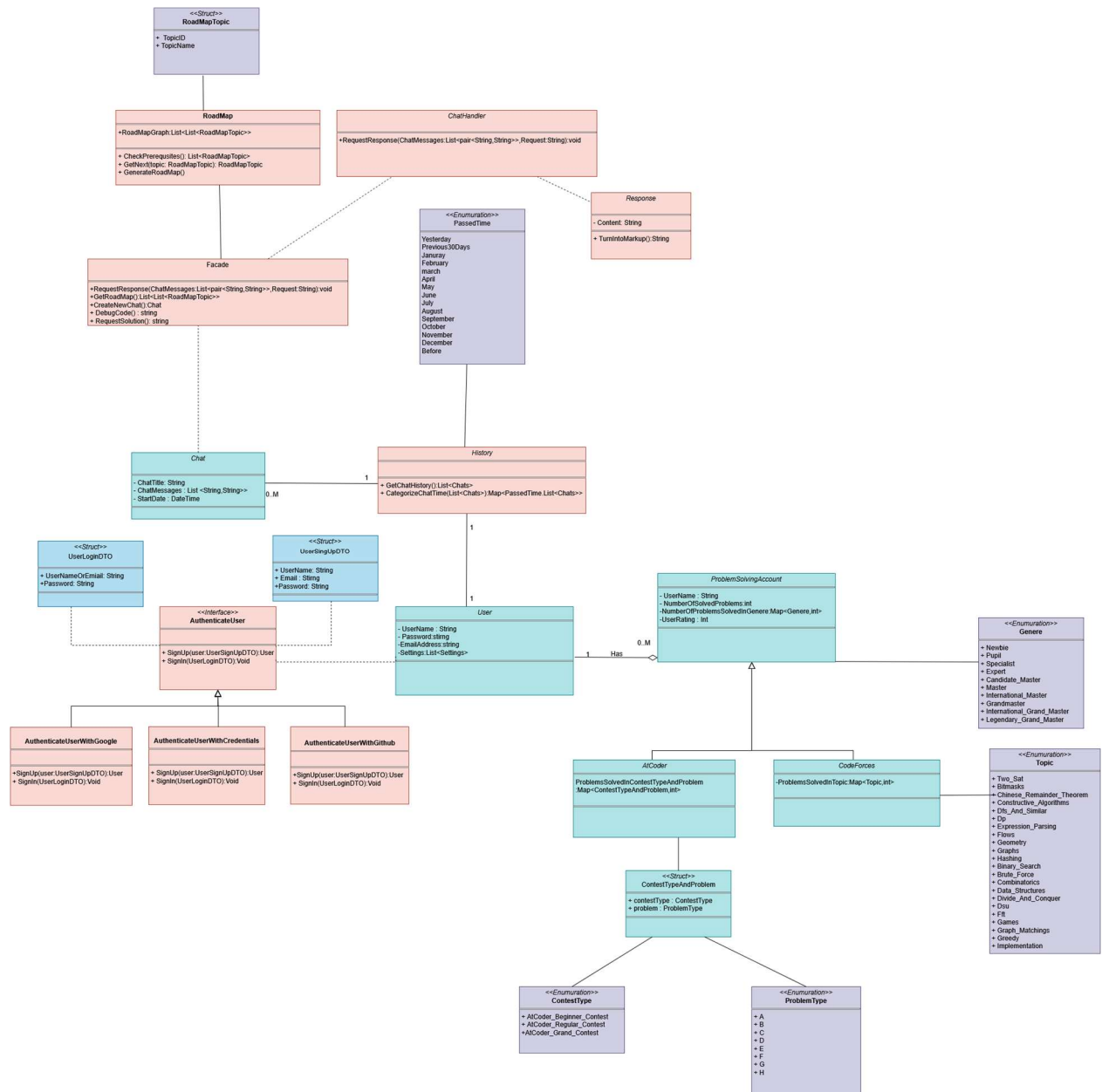


Figure 3-2:Class Diagram

3.8 Entity Relationship Diagram

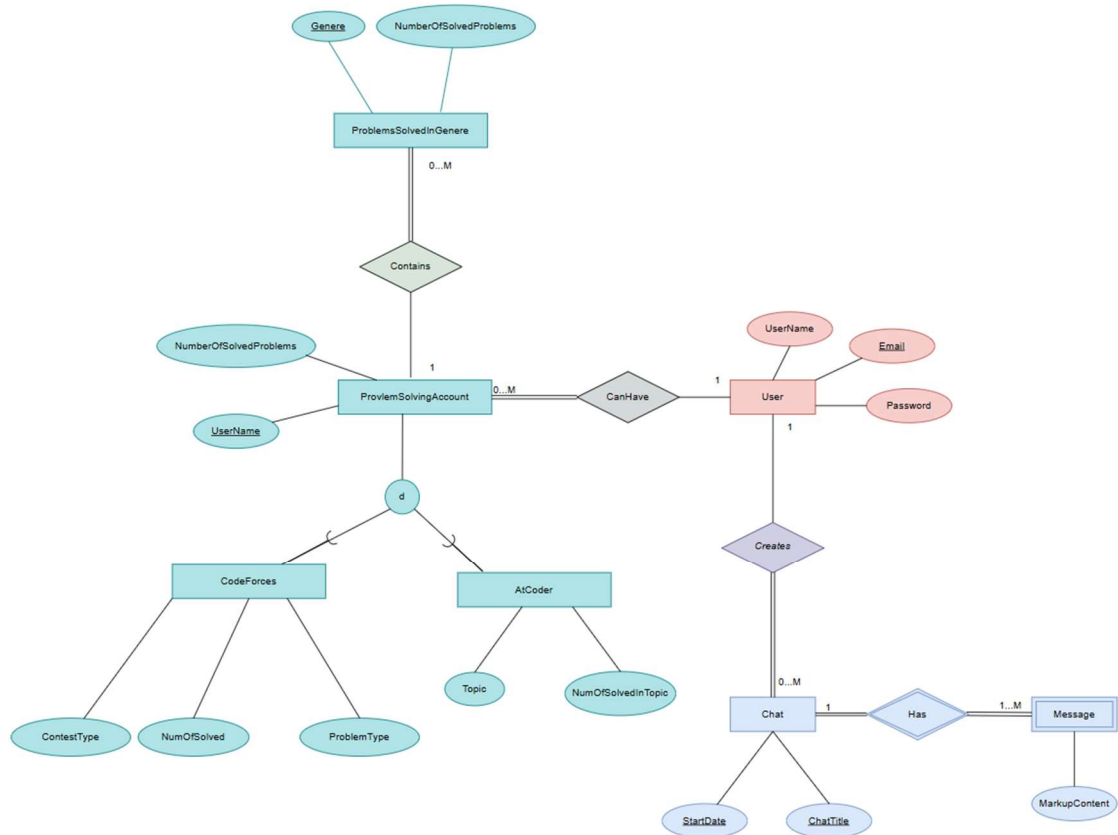


Figure 3-3:Entity Relationship Diagram

3.9 MongoDB Database Schema

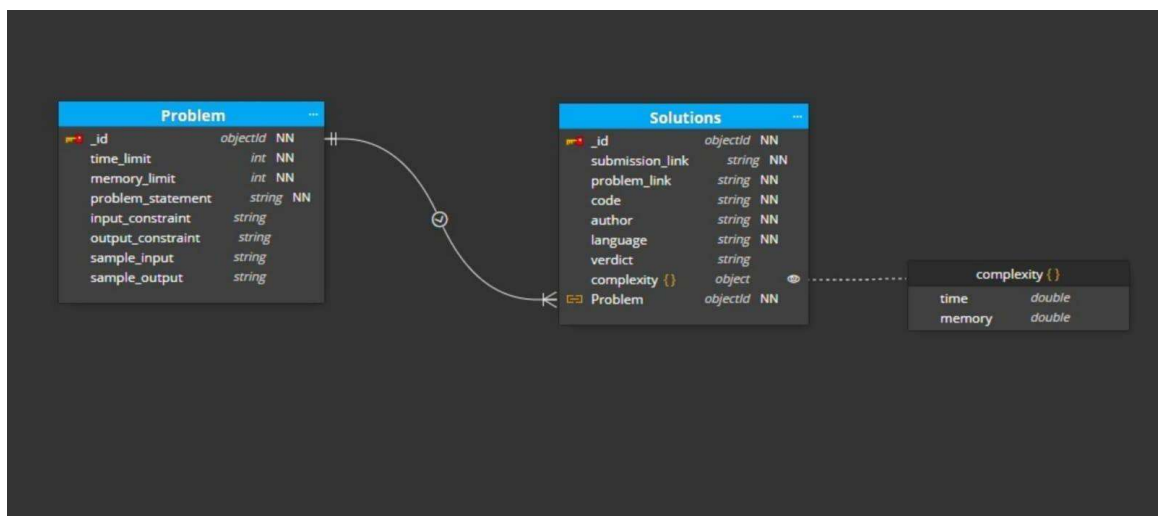


Figure 3-4:NOSql MongoDB Schema

3.9.1 Problems Collection Schema

Required Fields

- `time_limit` (int): The maximum allowed execution time for solving the problem (measured in milliseconds or seconds).
- `memory_limit` (int): The maximum memory usage permitted for solving the problem (measured in MB or KB).
- `problem_statement` (string): A detailed description of the coding problem.

Optional Fields

- `input_constraint` (string): Describes the limitations or restrictions on the input values.
- `output_constraint` (string): Defines the format or limitations for the expected output.
- `sample_input` (string): Provides example input values for a better understanding of the problem.
- `sample_output` (string): The expected output corresponding to the sample input.

3.9.2 Submissions Collection Schema

Required Fields

- `submission_link` (string): A URL or identifier linking to the submitted solution.
- `problem_link` (string): A reference to the problem that this submission is associated with.
- `code` (string): The actual source code of the submission.
- `author` (string): The name or ID of the person who submitted the solution.
- `language` (string): The programming language used for the submission.

Optional Fields

- `complexity`: A nested object containing computational complexity details:
 1. `time` (double): The estimated time complexity of the submitted solution.
 2. `memory` (double): The estimated memory usage of the submission.
- `verdict` (string): The result of the submission evaluation (e.g., "Accepted", "Wrong Answer", "Time Limit Exceeded").

3.10 Sequence Diagrams

3.10.1 Debug Code Sequence Diagram

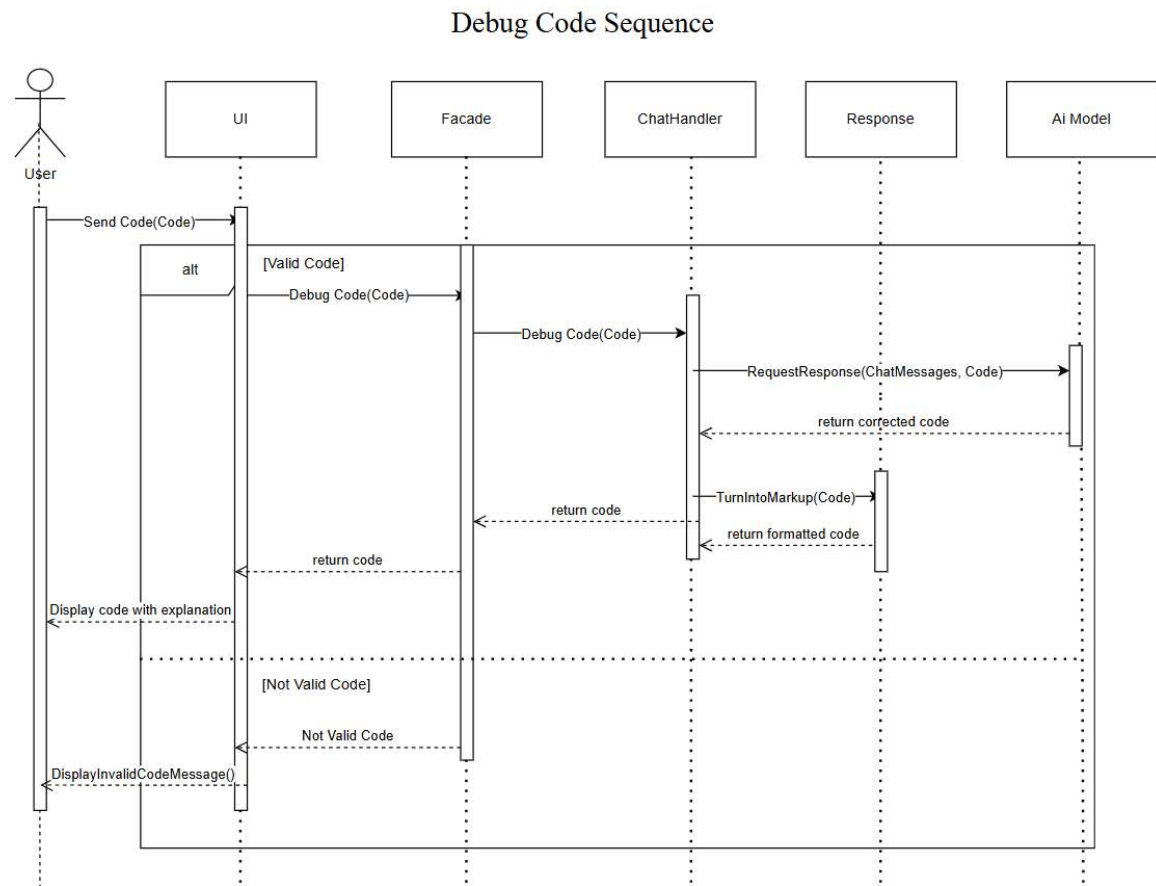


Figure 3-5: Debug Code Sequence Diagram

3.10.2 Request Roadmap Sequence Diagram

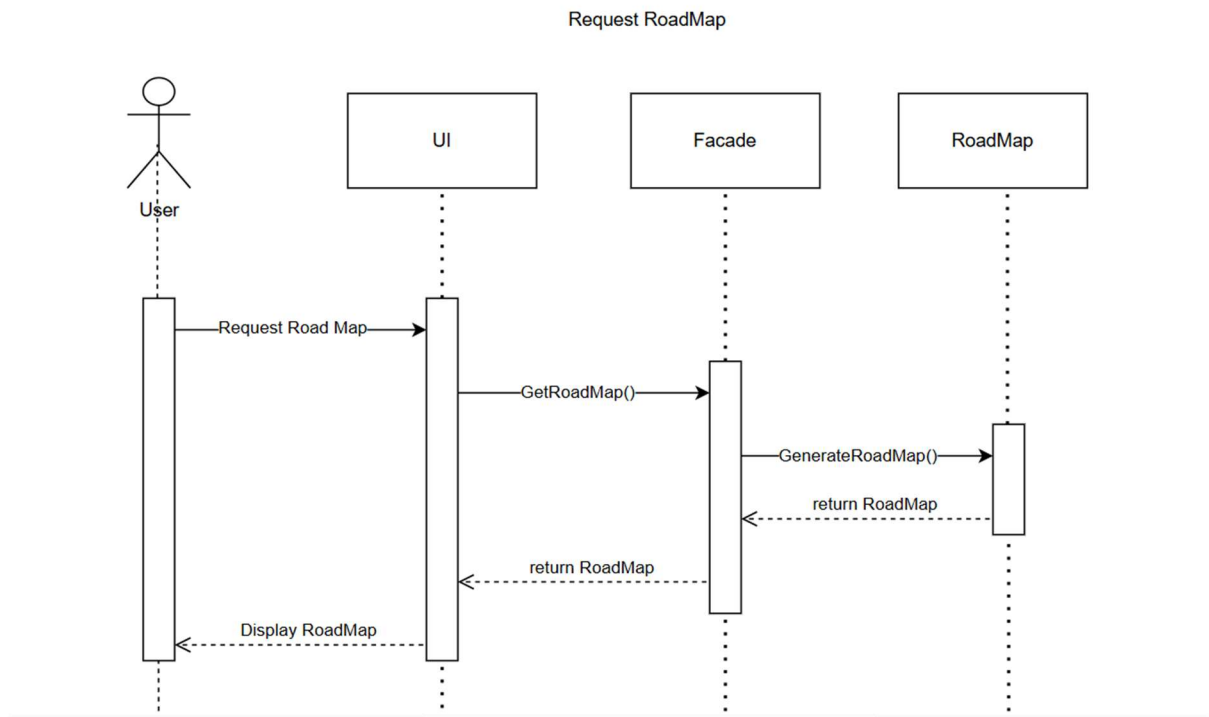


Figure 3-6: Request Roadmap Sequence Diagram

3.10.3 Request Solution Sequence Diagram

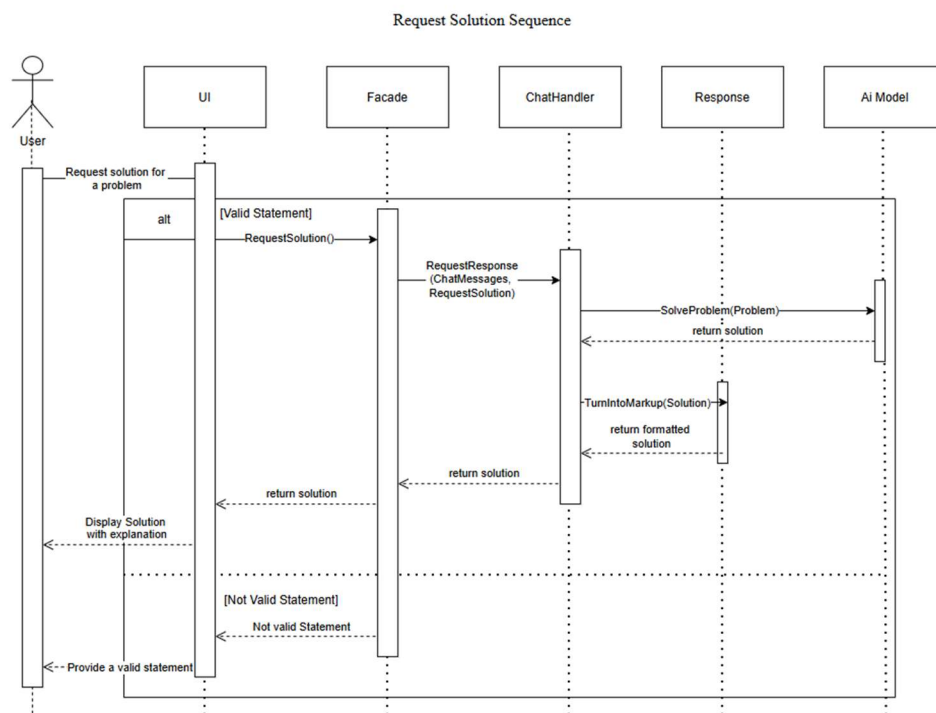


Figure 3-7: Request Solution Sequence Diagram

3.11 Use Case Narratives

Table 3-2: Register Use Case Table

Use Case ID:	Create Account	
Use Case Name:	Register	
Actors:	User	
Pre-Conditions:		
Post-Conditions:	The user successfully creates an account and can login to the system.	
Flow Of Events:	User Action	System Action
First Scenario:	1- User selects the “Create Account” option.	2- System displays options: “Sign up with Email”
	3- User selects “Sign up with Google Account.”	4- System prompts the user to enter Username, Email, Password, and Confirm Password.
Second Scenario	5-User enters details and submit the form.	6- System validates inputs and checks if the email is already registered
	7-If valid, the system creates the account and sends a verification email.	8- User verifies the email and completes registration

Exceptions:	User Action	System Action
	1- The user enters an already registered email address.	2- The system displays an error Message: "Email address is already in use."
	3-The user provides an invalid or weak password.	4-The system displays an error Message: "Password must meet security requirements."
	5-User fails to authenticate on Google or GitHub	6-The system displays an error Message: "Authentication failed. Please try again."

Table 3-3:Login Use case Table

Use Case ID:	Login	
Use Case Name:	Login	
Actors:	User	
Pre-Conditions:	The user has already created an account using Email, Google, or GitHub.	
Post-Conditions:	If login is successful, the system redirects the user to the Chabot's main page.	
Flow Of Events:	User Action	System Action
	1-User navigates to the login page.	2-System displays the login form.
	3-User enters a username and password.	4-System validates the input fields.
		5-System checks credentials in the database.
	3-User selects “Login with Google” or “Login with GitHub”.	4-System redirects the user to Google or GitHub authentication
	5-User grants Google or GitHub Account access.	6- System verifies Google or GitHub credentials.
		7-If authentication is successful, the system logs in the user and redirect him to the Chabot's main page.

Exceptions:	User Action	System Action
	1-User enters incorrect username or password.	2-System displays an error message: "Invalid credentials, please try again."
	3-User leaves the username or password field empty.	4-System asks the user to fill in all required fields.
	5-User forgets their password.	6-System provides a "Forgot Password" link for password recovery.
	7-Server is down or database is unreachable	8-System displays an error message: "Service unavailable"

Table 3-4: Request RoadMap Use Case Table

Use Case ID:	Request a roadmap	
Use Case Name:	Request a roadmap	
Actors:	User	
Pre-Conditions:	The user must be logged in.	
Post-Conditions:	The user receives a personalized roadmap for learning new algorithms and improving their problem-solving skills.	
Flow Of Events:	User Action	System Action
	1- The user asks the ChatBot for a roadmap for developing his skills in problem-solving.	2- The system retrieves the user's problem-solving stats from the database.
		3-The ChatBot generates a structured roadmap based on the user's stats.
		5- The ChatBot displays the roadmap in a structured format (weekly goals with specific topics and problems).

Table 3-5: Request a solution (using Text) use case Table

Use Case ID:	Request a solution	
Use Case Name:	Request a solution To an Entered Problem	
Actors:	User	
Pre-Conditions:	1 - The user must be logged in. 2 - The user must provide the problem they want to solve.	
Post-Conditions:	The user receives a detailed solution for the specified problem	
Flow Of Events:	User Action	System Action
	1- The user asks the ChatBot for a solution for a specific problem and enters the problem statement.	
	2-User optionally specifies the time or memory constraints.	3-The ChatBot processes the request and starts solving the problem.
		4-The ChatBot generates a solution (code and explanation) and checks its correctness by running test cases.
		5-The ChatBot displays the solution to the user, including the code, explanation, and test case results.
Exceptions:	User Action	System Action
	1-User does not provide a valid Statement.	2-The ChatBot responds with a message: "Please provide a valid complete problem statement."

Table 3-6: Convert algorithm to code Use case table

Use Case ID:	Convert algorithm to code	
Use Case Name:	Convert algorithm to code	
Actors:	User	
Pre-Conditions:	1 - The user must be logged in. 2 - The user must provide a detailed algorithm description or pseudo code.	
Post-Conditions:	The user receives a working implementation of the provided algorithm in the specified programming language.	
Flow Of Events:	User Action	System Action
	1-User asks the ChatBot to convert a specific algorithm to code.	2-The ChatBot prompts the user to input the algorithm details (pseudocode or textual description).
	3-User provides the algorithm description and specifies the desired programming language.	4-System validates the algorithm description and ensures it is sufficient to generate the code.
		5-System processes the algorithm, converts it to code, and ensures the code adheres to the constraints.
		6-System tests the generated code with example inputs (if provided).
		7-The ChatBot displays the code to the user, along with an example test cases and results.

Exceptions:	User Action	System Action
	1-User provides an incomplete or unclear algorithm description.	2-The ChatBot responds with a message: “The algorithm description is incomplete or Unclear. Please provide more details.”
	3-User requests an unsupported programming language	4- The ChatBot responds with a message”The specified programming language is not supported. Please choose another language.”

Table 3-7: Request Code Debug And/ Or Correction use case table

Use Case ID:	Debug a code	
Use Case Name:	Debug And/or Correct a code	
Actors:	User	
Pre-Conditions:	1 - The user must be logged in. 2 - The user must provide the code to be debugged.	
Post-Conditions:	The user receives the corrected and debugged version of the provided code, along with explanations of the fixes.	
Flow Of Events:	User Action	System Action
	1-User asks the ChatBot to debug a specific code.	2-The ChatBot prompts the user to upload or paste the code that needs debugging.
	3-User provide the code and specifies details (expected functionality, errors encountered).	4-System analyses the code and identifies potential issues (syntax, logic, runtime).
		5-System generates test cases and runs the code against it and detects errors.
		6-System corrects the identified issues optimize the code (if applicable), and tests the updated code.
		7-The ChatBot displays the debugged code to the user along with an explanation of the corrections.

Exceptions:	User Action	System Action
	1-User provides incomplete code.	2-The ChatBot responds with a message: "The provided code is incomplete. Please provide complete code."
	3-User provides code in an unsupported programming language.	4- The ChatBot responds with a message "The specified programming language is not supported. Please choose another language."

3.12 Gantt chart

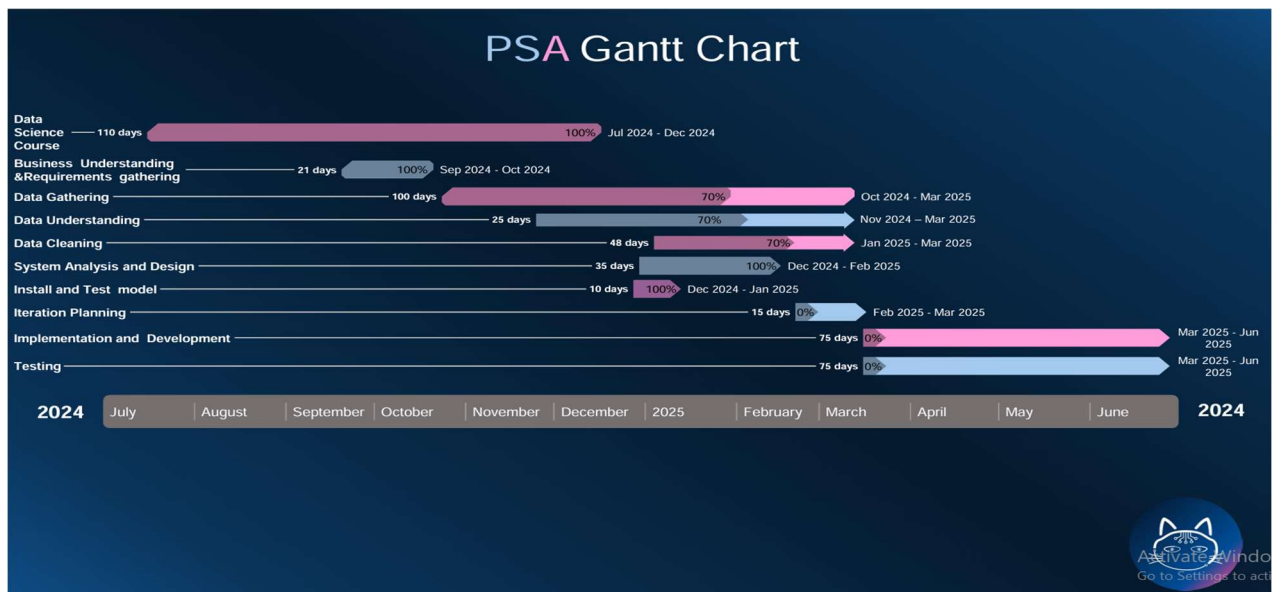


Figure 3-8: Project TimeLine

As shown in the Gantt chart, we started this project by taking a 110-day course in data science from July to December 2024. Meanwhile, we conducted business understanding and requirements gathering over a period of 21 days between September and October 2024.

Next, we initiated the data gathering phase, which spans from October 2024 to March 2025, covering a total of 100 days. Alongside, we worked on data understanding starting in November 2024, which is set to last for 25 days. The data cleaning phase, which involves processing raw data, started in January 2025 and is expected to take 48 days.

Once we had the necessary data insights, we moved on to system analysis and design, which lasted for 35 days from December 2024 to February 2025. The installation and testing of the model were completed in 10 days, running from December 2024 to January 2025.

Currently, we are in the iteration planning phase, which is scheduled for February to March 2025. The upcoming phases include implementation and development, which is planned for 75 days from March to June 2025, followed by testing, also scheduled for the same duration.

4 Chapter (4) PSA Design

4.1 UI Design

4.1.1 Mobile UI Design

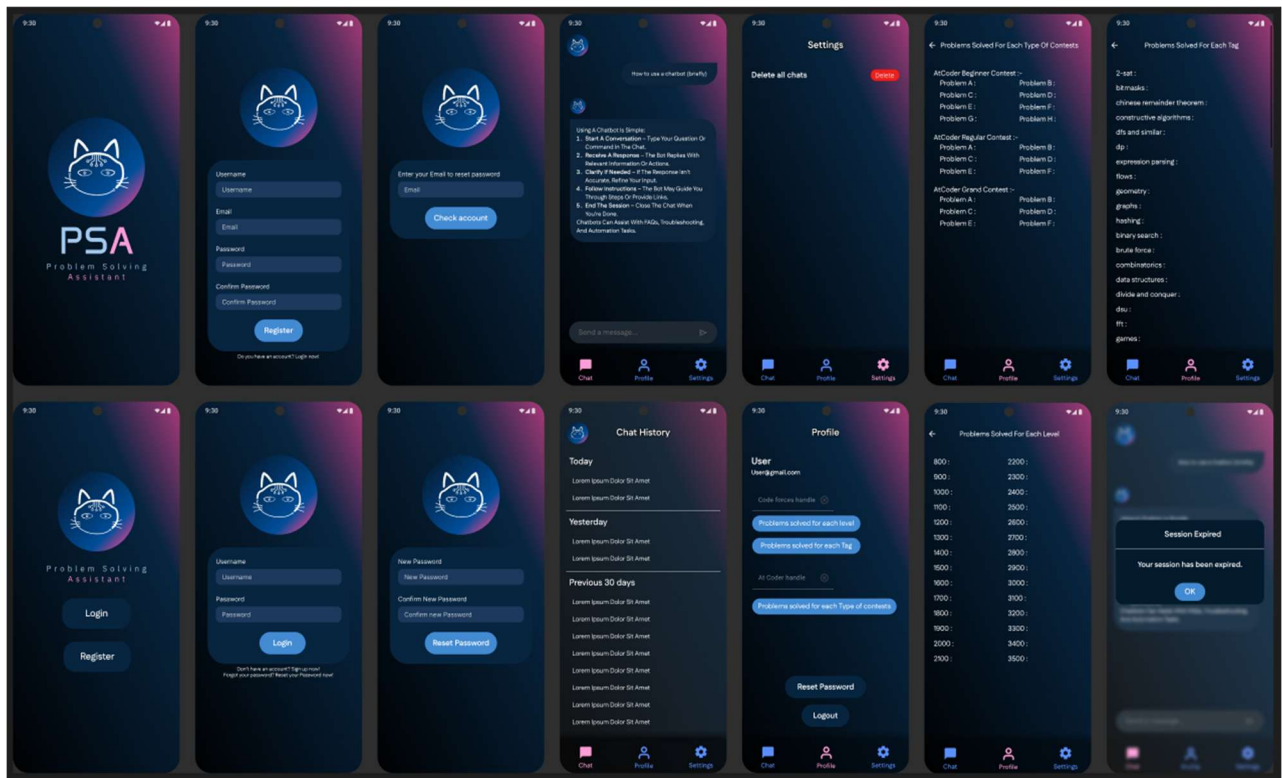


Figure 4-1: Mobile UI Design

4.1.2 Web UI Design

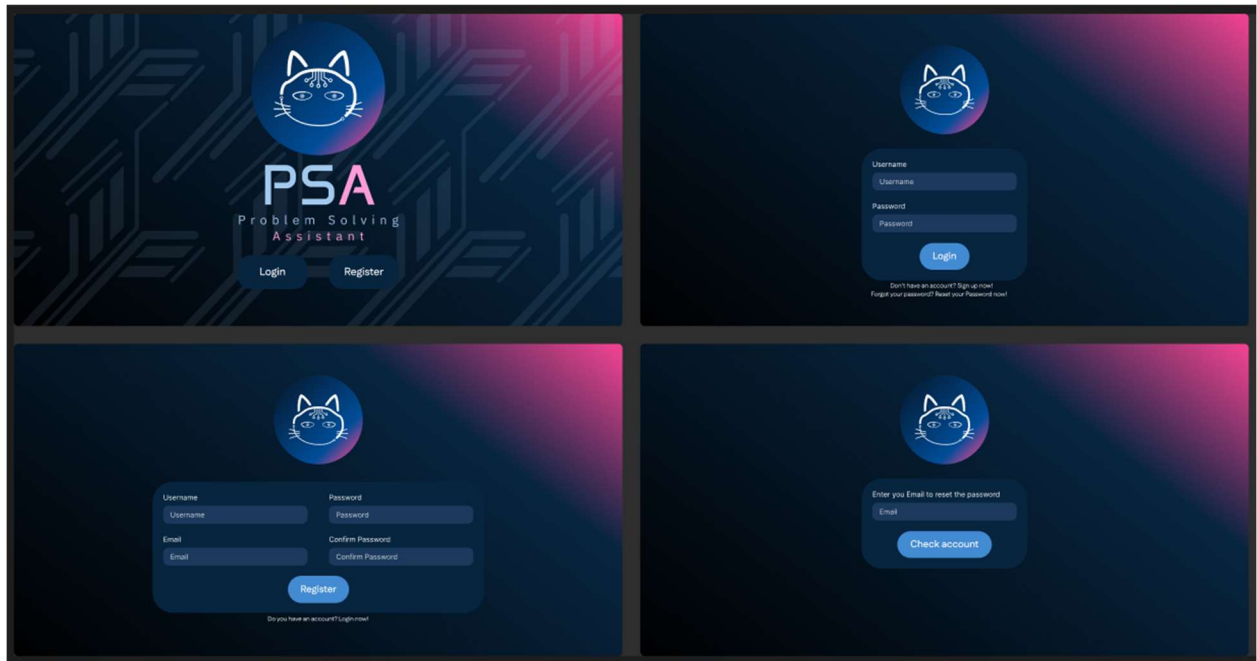


Figure 4-2: Web Home Screen, Register, Login, and Check Email

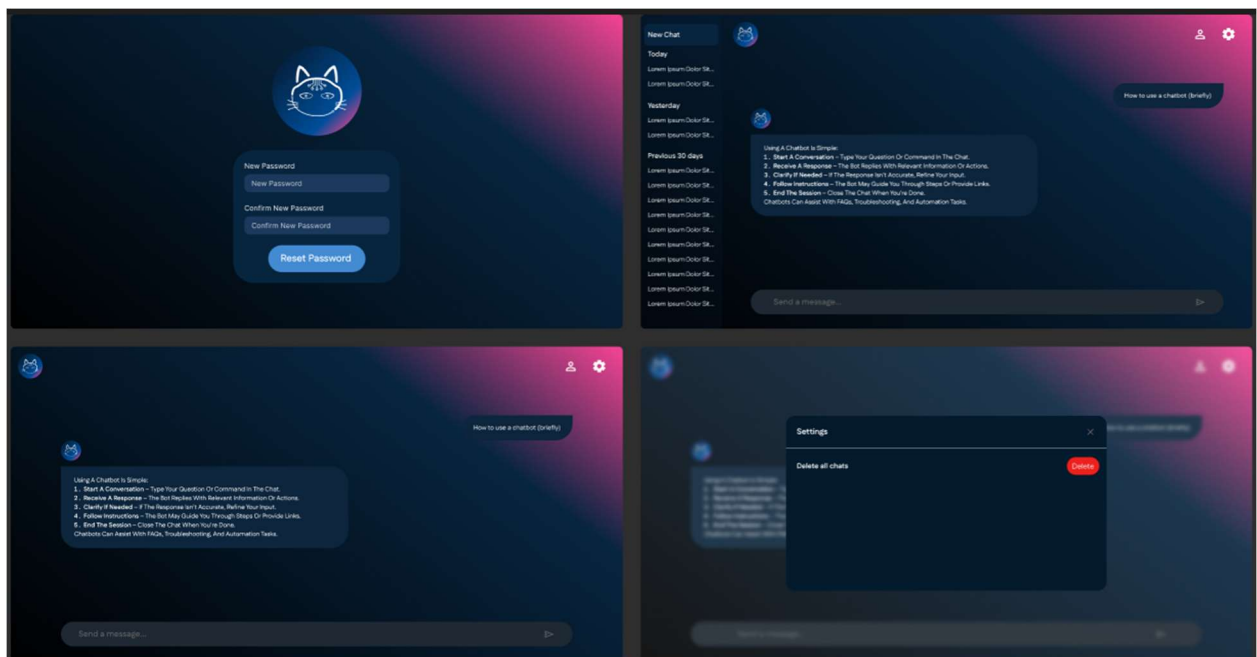


Figure 4-3: Web Reset Password, Chat Pages, setting

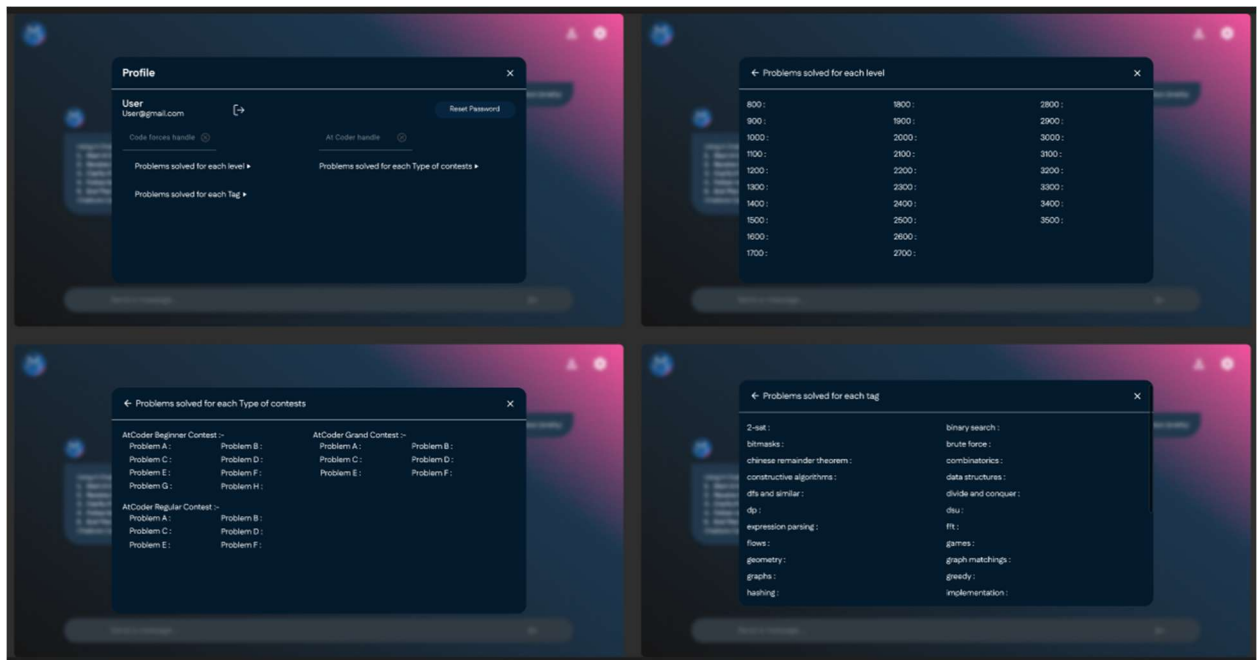


Figure 4-4: Web User Profile, and User statistics

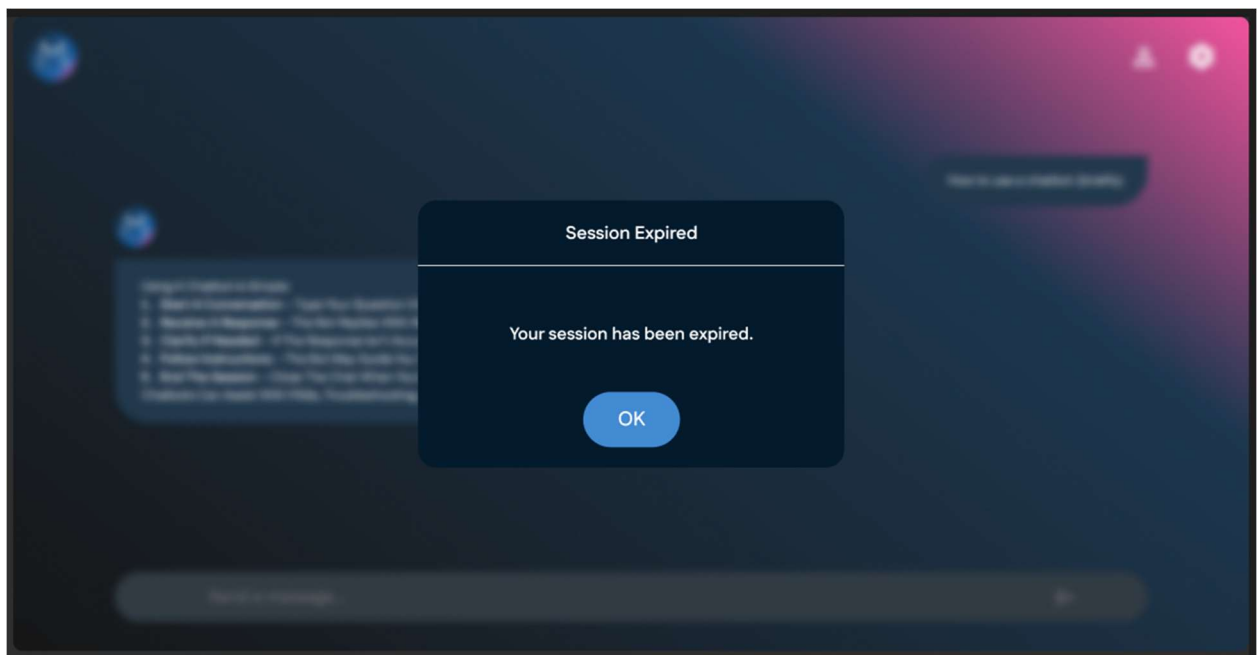


Figure 4-5: Web Session Expired

4.2 Implementation

After finishing the analysis and design, the requirements and details are clear enough to start the implementation. The implementation is divided into 3 parts: the backend, front end, and models retraining and integration. Throughout this chapter, will go through the process of implementing the main features which are, asking for roadmap, asking for problem solution, asking for debugging a code, asking for hints to solve the problem. We will show which models we have used, give a brief explanation of the main parts of the back end, and how the front-end looks.

4.2.1 Asking for a roadmap

This feature enables users to access a dedicated, pinned chat designed specifically for inquiries related to the learning roadmap. Users can ask questions such as the recommended sequence of topics, what to study next after completing a certain concept, and how to navigate through their learning journey (see **Figure 4-6**).

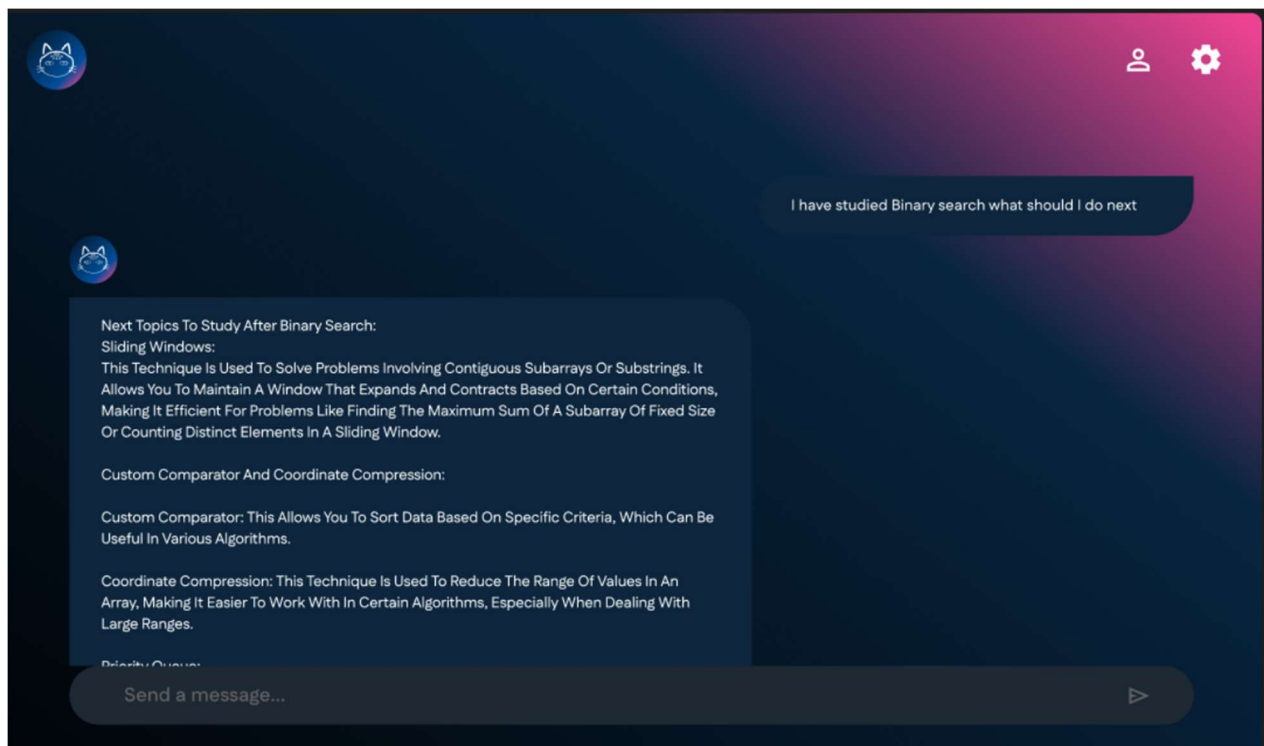


Figure 4-6: Asking for consequent topic

4.2.1.1 Roadmap Agents

To power this feature, we implemented a Retrieval-Augmented Generation (RAG) system utilizing trusted educational resources in the field of competitive programming. The RAG agent provides personalized guidance by generating a tailored roadmap based on user queries. Additionally, we maintain a history of the conversation by storing previous prompts and responses in a database. This allows the agent to leverage past interactions, enhancing context awareness and continuity in the conversation, ultimately improving the quality and relevance of the responses.

4.2.2 Ask for a solution

In this feature, the user can send a coding problem they found difficult to solve and request a solution from the model. Additionally, the user can specify the time and memory limits the solution must meet for it to be considered acceptable. This helps ensure that the provided solution aligns with the constraints of competitive programming platforms (Figure 4-7, 4-8, 4-9)

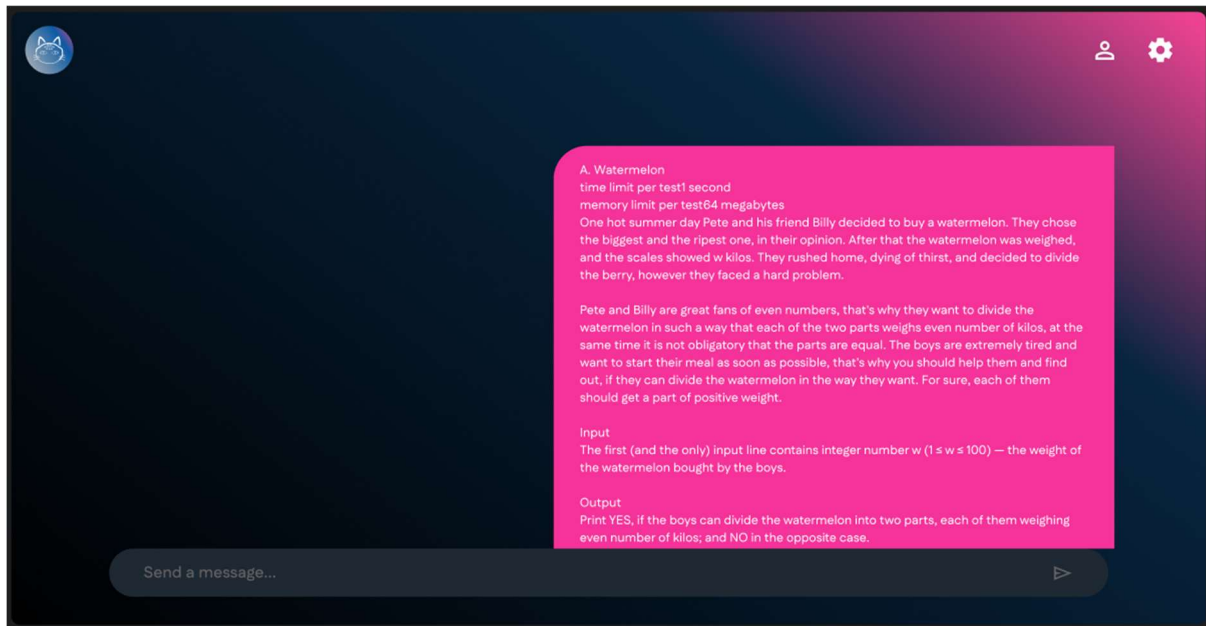


Figure 4-7: Asking for a problem solution

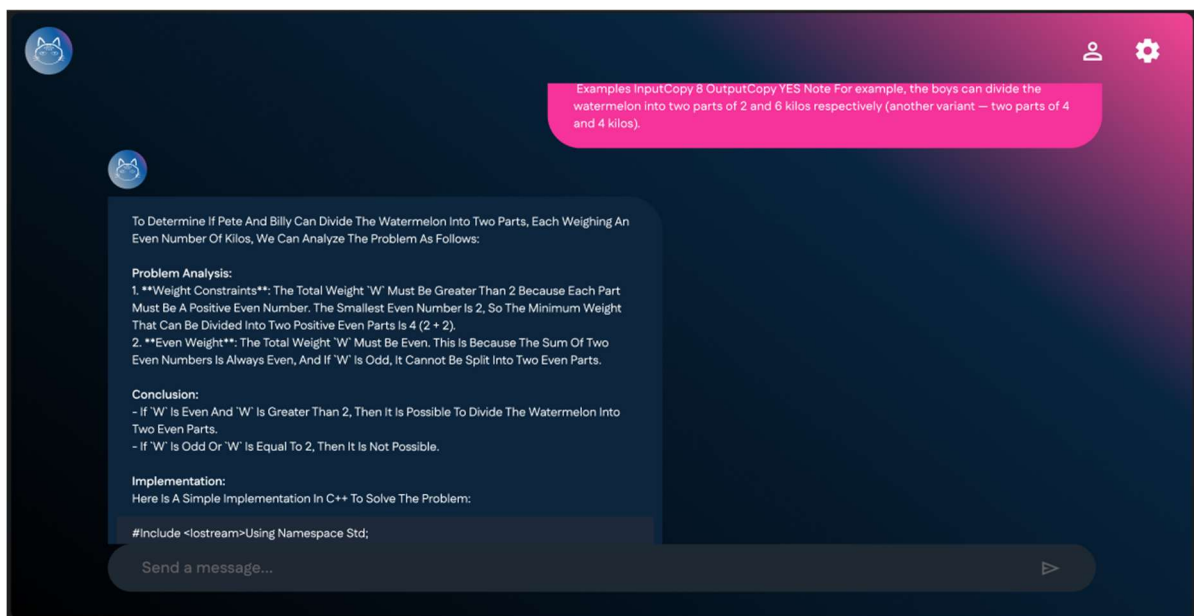


Figure 4-8: Chat response

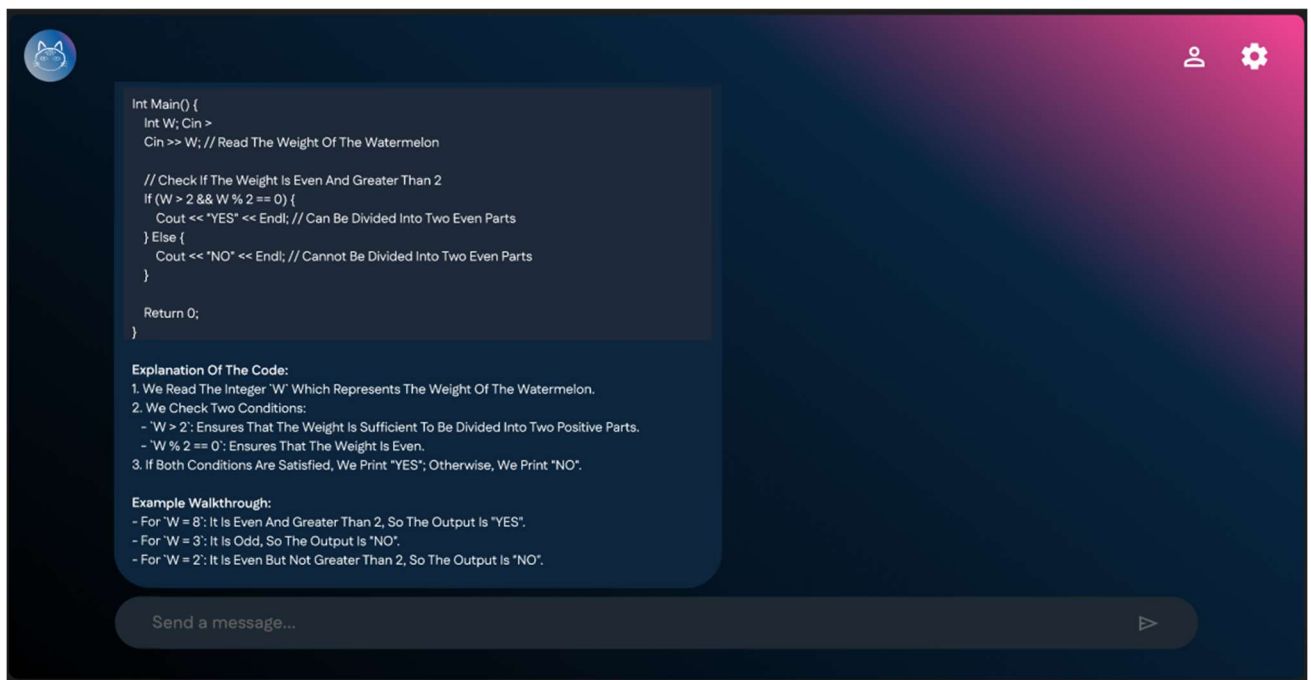


Figure 4-9 Rest of response

4.2.2.1 Solution Functionality

To power this feature, we implemented a Retrieval-Augmented Generation (RAG) system that leverages trusted educational resources in competitive programming. This system supplies the model with a solid base of algorithms, problem-solving strategies, and relevant examples, ensuring that the generated solutions are accurate and practical. By grounding the model's responses in verified materials, the RAG system effectively minimizes hallucination and enhances reliability.

4.2.3 Ask for a Code debug

In this feature, the user can submit code they have implemented to solve a specific problem but encountered issues with, whether these are syntax errors, runtime errors, or logical errors. The user then requests the model to analyse the code, identify the underlying problems, and attempt to fix them automatically. This allows users to efficiently debug their solutions and understand the reasons behind failures, helping them improve their coding skills while saving time.

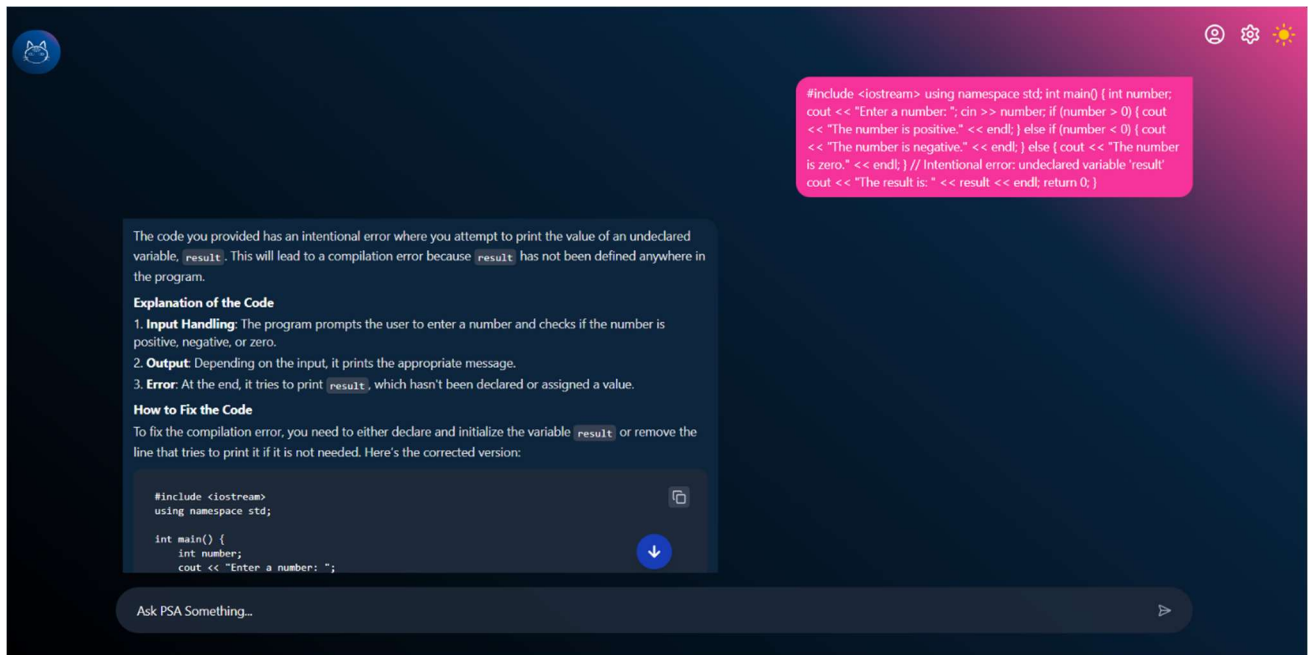


Figure 4-10 simple code needs to be debugged

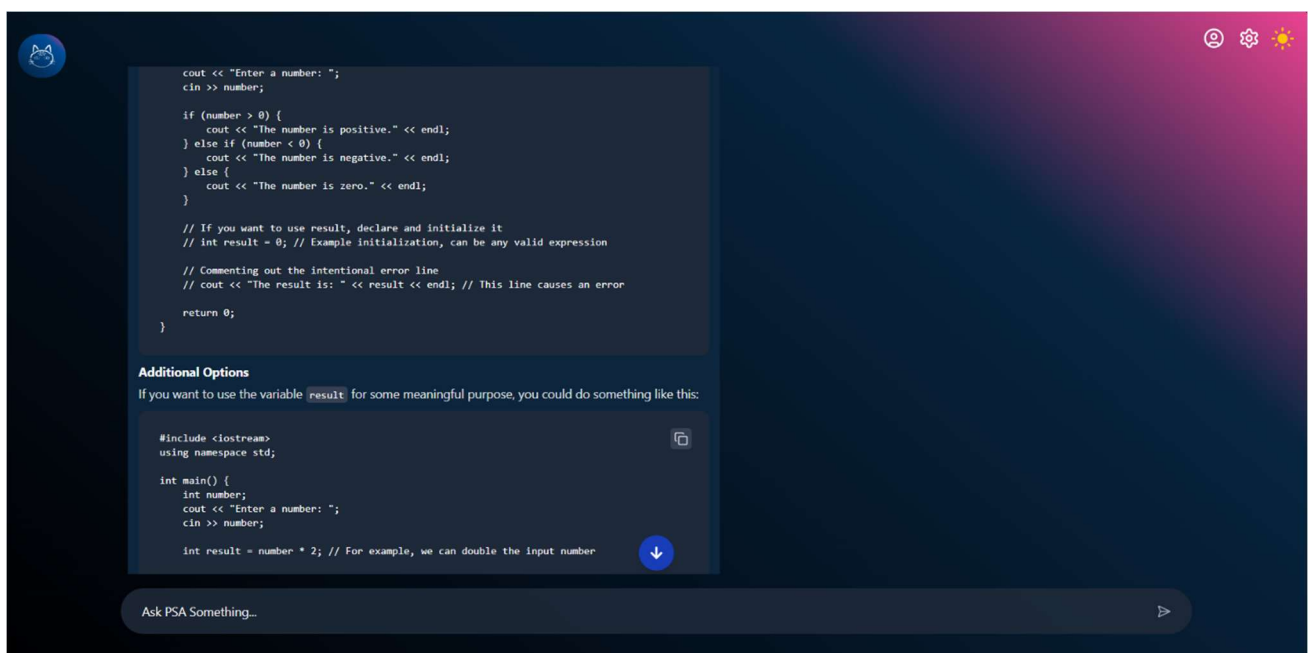


Figure 4-11 Response with debugged code

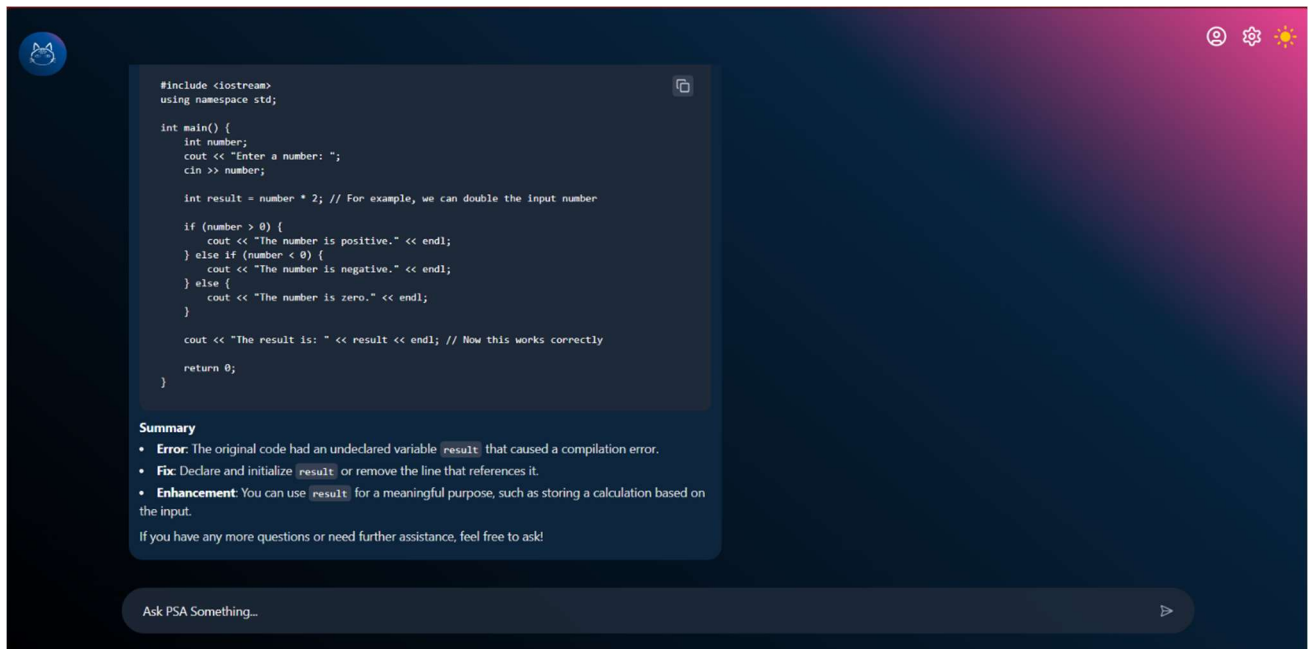


Figure 4-12 Rest of debugged code response

4.2.3.1 Debug Functionality

To power this feature, we implemented a Retrieval-Augmented Generation (RAG) system that leverages trusted educational resources and verified code examples in competitive programming. This system provides the model with a solid foundation in debugging practices, common error patterns, and language-specific pitfalls to enhance its ability to detect and correct issues accurately. By grounding the model's analysis in reliable materials, the RAG system minimizes hallucination while ensuring practical and effective debugging assistance for the user.

4.2.4 Online Judge Insights Integration

This feature is designed to collect and present detailed insights from popular competitive programming platforms, namely Codeforces and AtCoder, allowing users to track their progress and receive personalized assistance from the Roadmap Agent. In future iterations, a dedicated Insights Agent can be introduced to analyse this data dynamically and provide users with strategic guidance based on their performance patterns

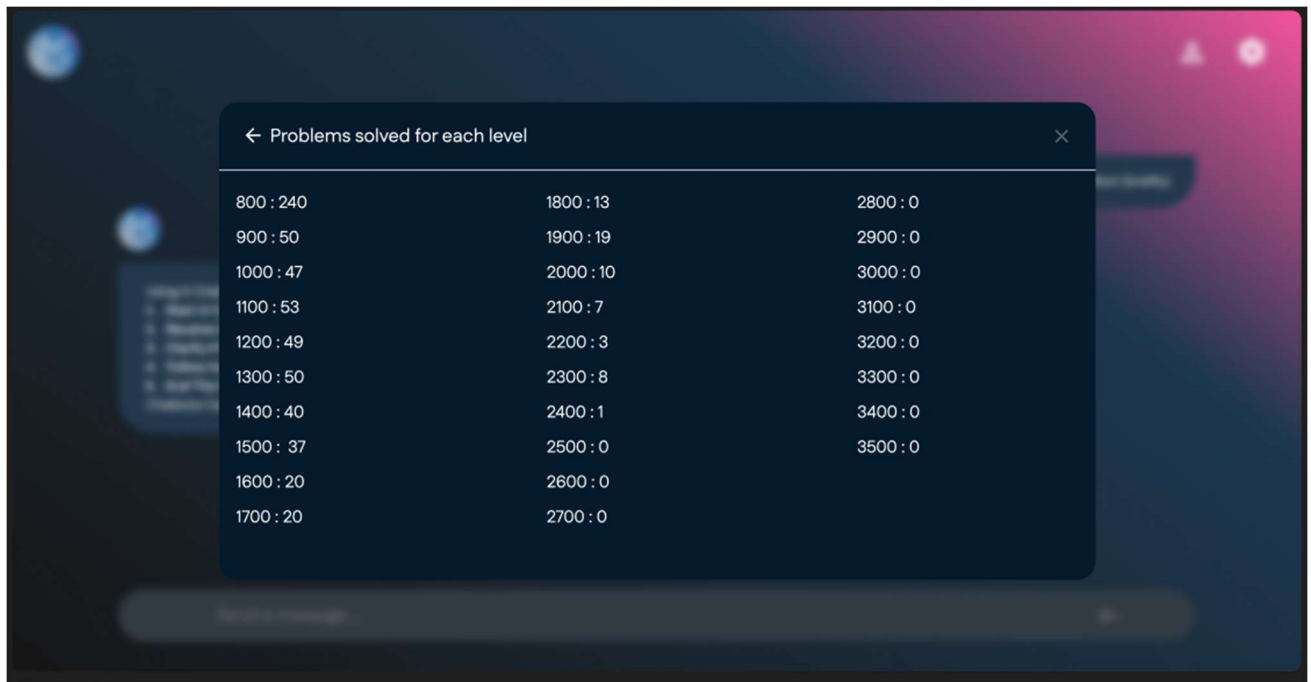


Figure 4-13 Codeforces rates

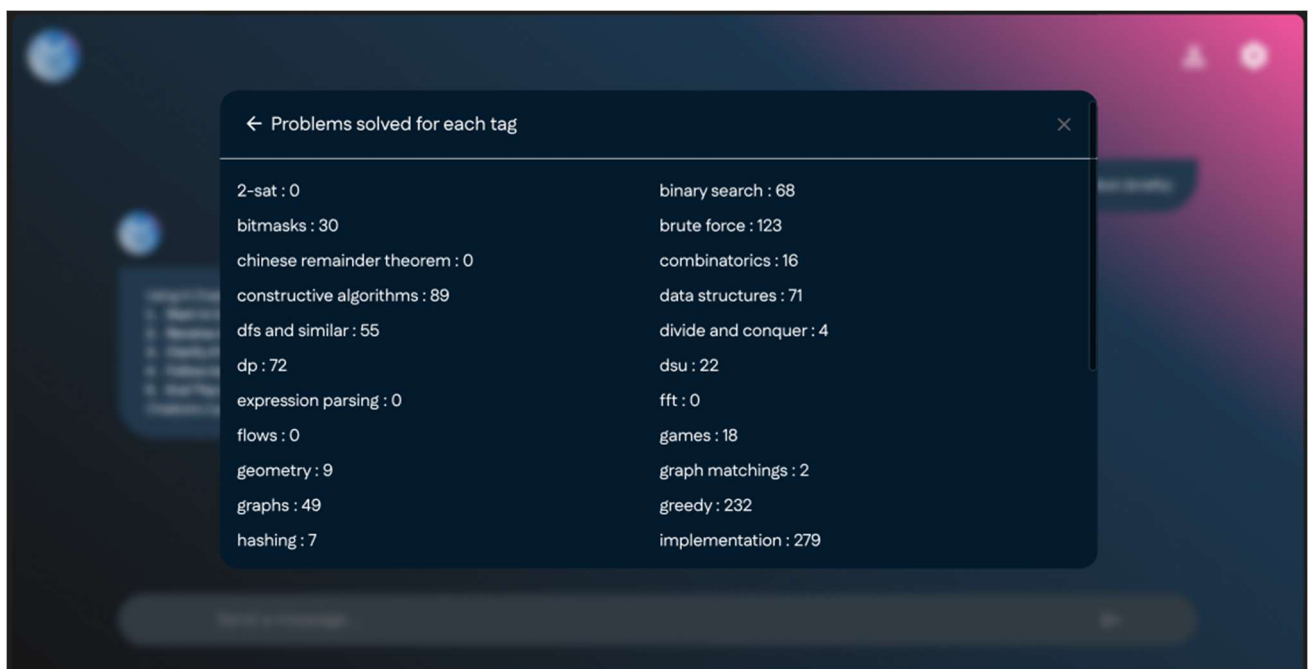


Figure 4-14 Codeforces topics

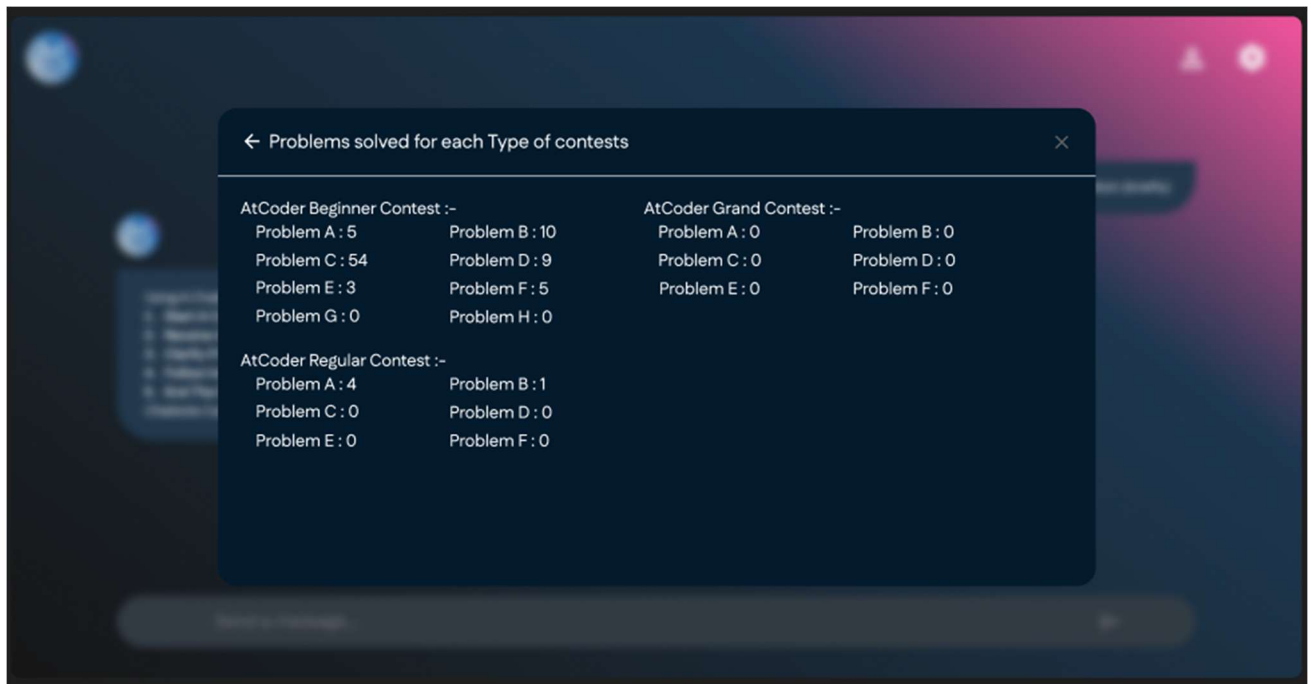


Figure 4-15 Atcoder insights

4.2.4.1 Handle Linking and Verification

To ensure secure integration, users are required to link their online judge handles via the platform. When a handle is submitted, it is stored in the database alongside the user's ID with a "pending verification" status. To verify authenticity and preserve user privacy, a randomized verification problem is generated. The user is then asked to submit a compilation error for this problem on the respective platform. This mechanism allows the system to validate ownership of the handle without compromising security or requiring platform credentials.

4.2.4.2 Insights Collection and Caching

Once verification is complete, the system retrieves user activity and performance data using the official APIs of Codeforces and AtCoder. The data is processed and filtered to extract meaningful insights, such as solved problems, contest performance, submission history, and difficulty trends. To ensure responsiveness and reduce dependency on external services, these insights are stored in a local cache. This approach improves availability, reduces API load, and enables faster response times when the user requests performance evaluations or assistance through the Roadmap Agent.

4.3 Used Tools

Table 4-1: Python Usage

Python		An interpreted, high level programming language.
In All Phases ● Used in scraping data with specific attributes needed. ● Used in understanding and cleaning data. ● Will be used to implement the business logic. - Will be used to test business logic based on predefined test cases.		
Pros ● Easy to learn. ● Scalability– Python is a highly scalable language and is also much faster than other languages such as R, Stata, and Matlab. ● Wide choice of libraries– comes with a host of Data Science and Data Analytics libraries including Pandas, NumPy, SciPy, Scikit Learn, StatsModels, and many more. ● Active Python community– No matter what your issue is, you can always count on the Python ecosystem to help and support you. ● Options for visualization– rich visualization frameworks allow you to make sense of the data at hand and also visualize your findings through. ● Easy Model Deployment		Cons ● Although Python has some nice visualization libraries. But, compared to R, visualizations are usually more convoluted, and the results are not always so pleasing to the eye.

Table 4-2: : Flutter Usage

Flutter		A free and open-source mobile UI framework. It allows the creation of a native mobile application with only one codebase.
Phase four: Implementation		
used to develop the front end for our mobile App.		
Pros		Cons
Relatively fast development – it allows you to use the same code base for building separate iOS and Android apps, so it speeds up the whole development process Flexibility – its automatic reload makes it possible to make changes to the code and see the results immediately in the app preview, without the need to recompile the code. Full customisation & fast rendering – it gives control over every pixel on the screen and lets you overlay & animate graphics, video, text and controls without limits.		Flutter apps are quite large and heavy to start with – They occupy a lot of space and take longer to download or update.

Table 4-3 : DrawIo Usage

Draw IO A free online diagramming tool used for creating flowcharts, UML diagrams, network diagrams, and more.	
In All Phases Used for Drawing UML diagrams	
Pros - Free and easy-to-use diagramming tool with cloud storage integration. - Supports collaboration and multiple export formats.	Cons - Limited offline functionality in the free version - Lacks some advanced features available in paid alternatives.

Table 4-4: MongoDB Usage

MongoDB A NoSQL document-based database that stores data in JSON-like format, offering high flexibility and scalability.	
From Data Collection to feedback Used for storing and Managing the AI model data efficiently.	
Pros Schema-less structure allows flexibility in data storage	Cons Requires additional indexing strategies for performance optimization.

Table 4-5: PostgreSQL Usage

PostgreSQL	
A powerful, open-source, relational database system known for reliability, extensibility, and performance	
Phase four: Implementation Will be used for relational database management For the ChatBot system.	
Pros - High-performance and supports complex queries efficiently. - ACID-compliant and supports transactions.	Cons Requires fine-tuning for optimal performance in large-scale applications.

Table 4-6: Figma Usage

Figma	
A browser-based UI and UX design application, with excellent design, prototyping, and code-generation tools	
Phase three: Analysis Figma was used to create our prototype.	
Pros Easy to use interface. Collaboration – you can share the design with your team members, can all log in at once, simultaneously making changes to it and all changes will be saved.	Cons Bad Transitions – Zooming in and out of designs is always a frustrating experience

Table 4-7: GitHub Usage

GitHub A web-based, code hosting platform used for version control and collaboration. It lets you and others work together on projects from anywhere.	
Phase two: Data pre-processing Phase four: Implementation GitHub used and will be used to share and work on any code.	
Pros Remote team collaboration – It simplifies the process of working with other people and makes it easy to cooperate on projects. Easy way to share changes – Team members can work on files and easily merge their changes with the master branch of the project.	Cons Steep learning curve- it needs time to know how to create a repository, make branches, pull, and merge your work with your team.

Table 4-8: Google Drive Usage

Google Drive		A file storage and synchronization service developed by Google.
Used in all phases All our documentation, presentations, data or even codes are stored and accessed by all of us with google drive.		
Used In the requirement Validation Creating the survey form and accessing it.		
Pros Easy to use interface – you can Create a new document, view shared folders between computers, view documents shared outside of your personal drive, see recent, starred, or deleted documents Microsoft office compatible – all Microsoft derivatives doc can be uploaded, shared, and can be accessed by anyone with access to the drive where the file is stored.		Cons Limited Storage – it only allows up to 15 gigabytes. Costly – In case you need extra space you must pay.

Table 4-9 : Zoom Usage

Zoom		A Video telephony and online chat services through a cloud-based peer-to peer software platform	
Used in all phases			
All our online meetings are on zoom.			
Pros		Cons	
Easy to use interface – you can create, access, chat, or call with an easy and user-friendly manner. Adaptability – can use it anywhere and on any device. Screen Sharing Clear audio Date Consumption – compared to other online meeting applications it's convenient.		- Limited amount time – each meeting is limited to 40 minutes. - Costly – In case you need extra time you must pay.	

Table 4-10: Microsoft PoweBI usage

Microsoft Power BI An Interactive data visualization and analytics tool for business intelligence (BI)	
Phase 1 : Analysis Phase Used to analyze the responses of the survey.	
Pros Excel Integration Custom Visualization – Offers a wide range of custom visualization Affordable	Cons Crowded User Interface – the canvas is very small that all the charts are crowded in a small space and makes it hard to see it.

Table 4-11: Django Usage

Django A high-level Python web framework that enables rapid development with a clean and pragmatic design	
Phase 1 : Analysis Phase Will serve as the backend framework for developing the web application APIs.	
Pros Built-in authentication, admin panel, and security features. Encourages rapid development- with the "batteries-included" approach.	Cons Crowded User Interface – the canvas is very small that all the charts are crowded in a small space and makes it hard to see it.

Table 4-12: Pycharm Professional Usage

Pycharm Professional A professional IDE for Python development that includes advanced features such as Django support, database tools, and code analysis.	
All The phases Will be the primary IDE for Python development. .	
Pros - Advanced debugging, database tools, and Django integration. - Excellent code intelligence and refactoring tools.	Cons - Paid version requires a subscription. - Can be resource-heavy on lower-end machines.

Table 4-13: React Usage

React A JavaScript library for building interactive user interfaces, primarily for single-page web applications.	
Phase four: Implementation Used for developing the front-end of our web applications.	
Pros Component-based architecture enables reusable UI elements. - Fast rendering with Virtual DOM.	Cons Requires additional libraries for state management (e.g., Redux).

Table 4-14: Hugging Face Usage

Hugging Face Transformer		An open-source platform providing state-of-the-art NLP models and AI tools for transformers, ChatBot, and machine learning research.	
Phase four: Implementation			
Used for integrating pre-trained AI models in our ChatBot and NLP applications.			
Pros		Cons	
Provides free pre-trained models for NLP and AI tasks. Active community and continuous updates		Some advanced features require a paid subscription. Large models require high computing power.	

Table 4-15: Selenium Usage

Selenium		A browser automation tool used for testing web applications and automating repetitive web tasks.	
Phase Five: Testing			
Used for automated testing of web applications.			
Pros		Cons	
- Supports multiple programming languages (Python, Java, etc.). - Useful for automated web scraping and testing.		- Requires browser drivers for different browsers. - Can be slow compared to API-based automation.	

Table 4-16: AnyDesk Usage

AnyDesk	
A remote desktop application that allows fast and secure access to other computers.	
Phase four: Implementation Used for remote collaboration, troubleshooting, and accessing development environments.	
Pros - Lightweight And fast remote access tool. - Secure Connection with encryption features.	Cons - Free version has limitations for commercial use - Performance may vary depending on network speed.

Table 4-17: DeepSeek Usage

DeepSeek	
An AI research lab specializing in deep learning models and innovative AI solutions.	
For the AI Model development Used as a Base Model for the AI Model development.	
Pros - Provides cutting-edge AI models and research insights. - Strong focus on deep learning and LLMs.	Cons Documentation and community support are still growing.

4.4 Courses

4.4.1 IBM Data Science Professional Certificate

This program is provided by IBM on Coursera platform. It consists of 10 courses. The program objective is to learn what data science is, the various activities of a data scientist's job, and methodology to think and work like a data scientist. Also, it helped us in applying various data science skills, techniques, and tools. It helped in developing our skills in python (analyse and visualisation data, and build and evaluate machine learning models and pipelines in python).

4.4.2 Flutter Course

From 2 Courses on Udemy platform and From Route. We have taken those Courses whose objective is to help us learn how to implement front end for cross platform mobile applications (Android & IOS) using Dart and Flutter.

4.4.3 Django Course

Using the Django documentation with prior knowledge at Backend, Design patterns, SOLID principles and some help of online recourses using Python

5 Chapter (5) Testing and Evaluation

5.1 Testing Objectives

The purpose of testing is to verify that all implemented features function correctly, are secure, handle edge cases gracefully, and meet the functional and non-functional requirements.

5.2 Testing Types and Tools

Layer	Testing Type	Tools Used
Backend (API)	Unit & Integration Tests	pytest, Django TestCase, DRF APIClient
Backend (API)	Manual Testing	Postman, curl
Backend (API)	Load Testing	Locust
Backend (API)	Security Testing	Manual OWASP Checklist
Frontend Web	Manual Testing	Browser DevTools, Postman
Frontend Mobile	Manual Testing	Android Emulator, Firebase Test Lab

5.3 Backend Test cases

5.3.1 Authentication Test cases

5.3.1.1 Registration Test cases

Field	Value
Test Case ID	TC_AUTH_01
Name	Register with valid data
Endpoint	/api/auth/register/
Method	POST
Input	{"first_name": "User", "last_name": "Test", "username": "user1", "email": "marawan.ah.ab@gmail.com", "password": "StrongPass@123"}
Expected	201 Created
Actual	201 Created
Result	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_02
<i>Name</i>	Register with invalid Email
<i>Endpoint</i>	/api/auth/register/
<i>Method</i>	POST
<i>Input</i>	{"first_name": "User", "last_name": "Test", "username": "user1", "email": "a@gmail.com", "password": "StrongPass@123"}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_03
<i>Name</i>	Register with invalid Password (no Symbol)
<i>Endpoint</i>	/api/auth/register/
<i>Method</i>	POST
<i>Input</i>	{"first_name": "User", "last_name": "Test", "username": "user1", "email": "marawan.ah.ab@gmail.com", "password": "strongpass@123"}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_04
Name	Register with invalid Password (no upper letters)
Endpoint	/api/auth/register/
Method	POST
Input	{"first_name": "User", "last_name": "Test", "username": "user1", "email": "marawan.ah.ab@gmail.com", "password": "strongpass@123"}
Expected	400 Bad Request
Actual	400 Bad Request
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_05
Name	Register with invalid Password (no lower letters)
Endpoint	/api/auth/register/
Method	POST
Input	{"first_name": "User", "last_name": "Test", "username": "user1", "email": "marawan.ah.ab@gmail.com", "password": "STRONGPASS@123"}
Expected	400 Bad Request
Actual	400 Bad Request
Result	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_06
<i>Name</i>	Register with invalid Password (no numeric)
<i>Endpoint</i>	/api/auth/register/
<i>Method</i>	POST
<i>Input</i>	{"first_name": "User", "last_name": "Test", "username": "user1", "email": "marawan.ah.ab@gmail.com", "password": "StrongPass@"}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_07
<i>Name</i>	Register with invalid Password (short password less than 8 char's)
<i>Endpoint</i>	/api/auth/register/
<i>Method</i>	POST
<i>Input</i>	{"first_name": "User", "last_name": "Test", "username": "user1", "email": "marawan.ah.ab@gmail.com", "password": "S@123"}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_08
<i>Name</i>	Register with invalid Password (no char's)
<i>Endpoint</i>	/api/auth/register/
<i>Method</i>	POST
<i>Input</i>	{"first_name": "User", "last_name": "Test", "username": "user1", "email": "marawan.ah.ab@gmail.com", "password": "@123"}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_09
<i>Name</i>	Register with invalid Password (null)
<i>Endpoint</i>	/api/auth/register/
<i>Method</i>	POST
<i>Input</i>	{"first_name": "User", "last_name": "Test", "username": "user1", "email": "marawan.ah.ab@gmail.com", "password": ""}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_10
<i>Name</i>	Register with invalid First name (null first_name)
<i>Endpoint</i>	/api/auth/register/
<i>Method</i>	POST
<i>Input</i>	{"first_name": "", "last_name": "Test", "username": "user1", "email": "marawan.ah.ab@gmail.com", "password": "StrongPass@123"}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_11
<i>Name</i>	Register with invalid First name (first_name contain numbers)
<i>Endpoint</i>	/api/auth/register/
<i>Method</i>	POST
<i>Input</i>	{"first_name": "test1", "last_name": "Test", "username": "user1", "email": "marawan.ah.ab@gmail.com", "password": "StrongPass@123"}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

<i>Field</i>	Value
<i>Test Case ID</i>	TC_AUTH_12
<i>Name</i>	Register with invalid First name (first_name contain symbols)
<i>Endpoint</i>	/api/auth/register/
<i>Method</i>	POST
<i>Input</i>	{"first_name": "test@", "last_name": "Test", "username": "user1", "email": "marawan.ah.ab@gmail.com", "password": "StrongPass@123"}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

<i>Field</i>	Value
<i>Test Case ID</i>	TC_AUTH_13
<i>Name</i>	Register with invalid Last name (null last_name)
<i>Endpoint</i>	/api/auth/register/
<i>Method</i>	POST
<i>Input</i>	{"first_name": "User", "last_name": "", "username": "user1", "email": "marawan.ah.ab@gmail.com", "password": "StrongPass@123"}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_14
<i>Name</i>	Register with invalid Last name (last_name contain numbers)
<i>Endpoint</i>	/api/auth/register/
<i>Method</i>	POST
<i>Input</i>	{"first_name": "test", "last_name": "Test1", "username": "user1", "email": "marawan.ah.ab@gmail.com", "password": "StrongPass@123"}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_15
<i>Name</i>	Register with invalid Last name (last_name contain symbols)
<i>Endpoint</i>	/api/auth/register/
<i>Method</i>	POST
<i>Input</i>	{"first_name": "test", "last_name": "Test@", "username": "user1", "email": "marawan.ah.ab@gmail.com", "password": "StrongPass@123"}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_16
Name	Register with invalid username (null)
Endpoint	/api/auth/register/
Method	POST
Input	{"first_name": "test", "last_name": "Test@", "username": "", "email": "marawan.ah.ab@gmail.com", "password": "StrongPass@123"}
Expected	400 Bad Request
Actual	400 Bad Request
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_17
Name	Register with invalid username (already registered in our database)
Endpoint	/api/auth/register/
Method	POST
Input	{"first_name": "test", "last_name": "Test@", "username": "user1", "email": "marawan.ah.ab@gmail.com", "password": "StrongPass@123"}
Expected	400 Bad Request
Actual	400 Bad Request
Result	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_18
<i>Name</i>	Register with invalid Email (null)
<i>Endpoint</i>	/api/auth/register/
<i>Method</i>	POST
<i>Input</i>	{"first_name": "test", "last_name": "Test@", "username": "user1", "email": "", "password": "StrongPass@123"}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_19
<i>Name</i>	Register with invalid Email (already registered in our database)
<i>Endpoint</i>	/api/auth/register/
<i>Method</i>	POST
<i>Input</i>	{"first_name": "test", "last_name": "Test@", "username": "user1", "email": "marawan.ah.ab@gmail.com", "password": "StrongPass@123"}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

5.3.1.2 Login Test cases

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_20
<i>Name</i>	Login with valid data
<i>Endpoint</i>	/api/auth/login /
<i>Method</i>	POST
<i>Input</i>	{"username": "user1" OR "email": "marawan.ah.ab@gmail.com", "password": "StrongPass@123"}
<i>Expected</i>	200 OK
<i>Actual</i>	200 OK
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_21
<i>Name</i>	Login with invalid username or email (null)
<i>Endpoint</i>	/api/auth/login /
<i>Method</i>	POST
<i>Input</i>	{"username": "" OR "email": "", "password": "StrongPass@123"}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_22
Name	Login with invalid username or email (already registered in our database)
Endpoint	/api/auth/login /
Method	POST
Input	{"username": "user1" OR "email": "marawan.ah.ab@gmail.com", "password": "StrongPass@123"}
Expected	400 Bad Request
Actual	400 Bad Request
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_23
Name	Login with invalid password (null)
Endpoint	/api/auth/login /
Method	POST
Input	{"username": "user1" OR "email": "marawan.ah.ab@gmail.com", "password": ""}
Expected	400 Bad Request
Actual	400 Bad Request
Result	Pass

5.3.1.3 OTP Test cases

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_24
<i>Name</i>	OTP with valid email
<i>Endpoint</i>	/api/auth/otp/
<i>Method</i>	POST
<i>Input</i>	{"email": "marawan.ah.ab@gmail.com"}
<i>Expected</i>	200 OK
<i>Actual</i>	200 OK
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_25
<i>Name</i>	OTP with invalid email (null)
<i>Endpoint</i>	/api/auth/otp/
<i>Method</i>	POST
<i>Input</i>	{"email": ""}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_26
<i>Name</i>	OTP with invalid email (not valid email using regex)
<i>Endpoint</i>	/api/auth/otp/
<i>Method</i>	POST
<i>Input</i>	{"email": "test.gmail.com"}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_27
<i>Name</i>	OTP with invalid email (not valid email using regex)
<i>Endpoint</i>	/api/auth/otp/
<i>Method</i>	POST
<i>Input</i>	{"email": "test@.com"}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

5.3.1.4 Verify OTP Test cases

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_28
<i>Name</i>	OTP check with correct OTP
<i>Endpoint</i>	/api/auth/verify_otp
<i>Method</i>	POST
<i>Input</i>	{"email": "test@.com"}
<i>Expected</i>	200 OK
<i>Actual</i>	200 OK
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_29
<i>Name</i>	OTP check with wrong OTP
<i>Endpoint</i>	/api/auth/verify_otp
<i>Method</i>	POST
<i>Input</i>	{"OTP" : "wrong OTP"}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_29
<i>Name</i>	OTP check with wrong OTP
<i>Endpoint</i>	/api/auth/verify_otp
<i>Method</i>	POST
<i>Input</i>	{"OTP" : "wrong OTP"}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

5.3.1.5 Check Email Test cases

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_30
<i>Name</i>	OTP check with wrong OTP
<i>Endpoint</i>	/api/auth/verify_otp
<i>Method</i>	POST
<i>Input</i>	{"OTP" : "wrong OTP"}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_31
<i>Name</i>	check Email invalid (null)
<i>Endpoint</i>	/api/auth/check_email
<i>Method</i>	GET
<i>Input</i>	{"email": ""}
<i>Expected</i>	200 OK
<i>Actual</i>	200 OK
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_32
<i>Name</i>	check Email invalid (not valid email using regex)
<i>Endpoint</i>	/api/auth/check_email
<i>Method</i>	GET
<i>Input</i>	{"email": "test@.com"}
<i>Expected</i>	200 OK
<i>Actual</i>	200 OK
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_33
<i>Name</i>	check Email invalid (not registerd)
<i>Endpoint</i>	/api/auth/check_email
<i>Method</i>	GET
<i>Input</i>	{"email": "marawan@gmail.com"}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

5.3.1.6 Reset Password Test cases

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_34
<i>Name</i>	reset password with valid password
<i>Endpoint</i>	/api/auth/reset_password/
<i>Method</i>	POST
<i>Input</i>	{"email": "marawan@gmail.com", "Password": "Marwan@312"}
<i>Expected</i>	200 OK
<i>Actual</i>	200 OK
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_35
<i>Name</i>	reset password with invalid password (null)
<i>Endpoint</i>	/api/auth/reset_password/
<i>Method</i>	POST
<i>Input</i>	{"email": "marawan@gmail.com", "Password": ""}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_36
<i>Name</i>	reset password with invalid password (password less than 8 char)
<i>Endpoint</i>	/api/auth/reset_password/
<i>Method</i>	POST
<i>Input</i>	{"email": "marawan@gmail.com", "Password": "Ma@312"}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_37
<i>Name</i>	reset password with invalid password (Password all char's Uppercase)
<i>Endpoint</i>	/api/auth/reset_password/
<i>Method</i>	POST
<i>Input</i>	{"email": "marawan@gmail.com", "Password": "MARWAN@312"}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_38
<i>Name</i>	reset password with invalid password (Password all char's Lowercase)
<i>Endpoint</i>	/api/auth/reset_password/
<i>Method</i>	POST
<i>Input</i>	{"email": "marawan@gmail.com", "Password": "marwan@312"}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_39
<i>Name</i>	reset password with invalid password (Password with no symbols)
<i>Endpoint</i>	/api/auth/reset_password/
<i>Method</i>	POST
<i>Input</i>	{"email": "marawan@gmail.com", "Password": "Marwan312"}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_40
<i>Name</i>	reset password with invalid password (Password with no numbers)
<i>Endpoint</i>	/api/auth/reset_password/
<i>Method</i>	POST
<i>Input</i>	{"email": "marawan@gmail.com", "Password": "Marwan@"}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_41
Name	reset password with invalid password (Password with no char)
Endpoint	/api/auth/reset_password/
Method	POST
Input	{"email": "marawan@gmail.com", "Password": "Marwan@"}
Expected	400 Bad Request
Actual	400 Bad Request
Result	Pass

5.3.2 User Management Test cases

<i>Field</i>	<i>Value</i>
Test Case ID	TC_USER_1
Name	update user with valid data
Endpoint	/api/user/update/
Method	PUT
Input	{"first_name": "Marwan", "last_name": "Ahmed"}
Expected	200 OK
Actual	200 OK
Result	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_USER_2
<i>Name</i>	update user with invalid data (first_name null)
<i>Endpoint</i>	/api/user/update/
<i>Method</i>	PUT
<i>Input</i>	{"first_name": "", "last_name": "Ahmed"}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_USER_3
<i>Name</i>	update user with invalid data (first_name contains number)
<i>Endpoint</i>	/api/user/update/
<i>Method</i>	PUT
<i>Input</i>	{"first_name": "Marwan123", "last_name": "Ahmed"}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Passs

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_USER_4
<i>Name</i>	update user with invalid data (first_name contains symbols)
<i>Endpoint</i>	/api/user/update/
<i>Method</i>	PUT
<i>Input</i>	{"first_name": "Marwan123", "last_name": "Ahmed"}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_USER_5
<i>Name</i>	update user with invalid data (first_name doesn't have char's)
<i>Endpoint</i>	/api/user/update/
<i>Method</i>	PUT
<i>Input</i>	{"first_name": "@312", "last_name": "Ahmed"}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_USER_6
Name	update user with invalid data (last_name null)
Endpoint	/api/user/update/
Method	PUT
Input	{"first_name": "@312", "last_name": "Ahmed"}
Expected	400 Bad Request
Actual	400 Bad Request
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_USER_7
Name	update user with invalid data (last_name contain numbers)
Endpoint	/api/user/update/
Method	PUT
Input	{"first_name": "@312", "last_name": "Ahmed"}
Expected	400 Bad Request
Actual	400 Bad Request
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_USER_8
Name	update user with invalid data (last_name contain symbols)
Endpoint	/api/user/update/
Method	PUT
Input	{"first_name": "@312", "last_name": "Ahmed"}
Expected	400 Bad Request
Actual	400 Bad Request
Result	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_USER_9
<i>Name</i>	update user with invalid data (last_name doesn't have char's
<i>Endpoint</i>	/api/user/update/
<i>Method</i>	PUT
<i>Input</i>	{"first_name": "Marwan", "last_name": "@312"}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

5.3.2.1 Add Handle

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_USER_10
<i>Name</i>	check Codeforces handle using valid handle
<i>Endpoint</i>	/api/user/add_cf_handle/
<i>Method</i>	POST
<i>Input</i>	{"handle": "Maro"}
<i>Expected</i>	200 OK
<i>Actual</i>	200 OK
<i>Result</i>	Pass

<i>Field</i>	Value
<i>Test Case ID</i>	TC_USER_11
<i>Name</i>	check Codeforces handle using invalid handle (not on cf databases)
<i>Endpoint</i>	/api/user/add_cf_handle/
<i>Method</i>	POST
<i>Input</i>	{"handle": "Maro"}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

<i>Field</i>	Value
<i>Test Case ID</i>	TC_USER_12
<i>Name</i>	check codeforces handle using valid handle
<i>Endpoint</i>	/api/user/add_atcoder_handle/
<i>Method</i>	POST
<i>Input</i>	{"handle": "Maro312"}
<i>Expected</i>	200 OK
<i>Actual</i>	200 OK
<i>Result</i>	Pass

<i>Field</i>	Value
<i>Test Case ID</i>	TC_USER_13
<i>Name</i>	check Codeforces handle using invalid handle (not on atcoder databases)
<i>Endpoint</i>	/api/user/add_atcoder_handle/
<i>Method</i>	POST
<i>Input</i>	{"handle": "Marozzzzzzzzzzzz"}
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

5.3.3 Chat Test cases

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_CHAT_1
<i>Name</i>	create chat
<i>Endpoint</i>	/api/chat/create/
<i>Method</i>	POST
<i>Input</i>	{"....."}
<i>Expected</i>	200 OK
<i>Actual</i>	200 OK
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_CHAT_2
<i>Name</i>	send prompt in scope
<i>Endpoint</i>	/api/chat/send_prompt/{chat_id}/
<i>Method</i>	POST
<i>Input</i>	{"prompt": "what do you know about binary search"}
<i>Expected</i>	200 OK
<i>Actual</i>	200 OK
<i>Result</i>	Pass

<i>Field</i>	Value
<i>Test Case ID</i>	TC_CHAT_3
<i>Name</i>	send prompt outside scope
<i>Endpoint</i>	/api/chat/send_prompt/{chat_id}/
<i>Method</i>	POST
<i>Input</i>	{"prompt": "who was Egypt first President"}
<i>Expected</i>	200 OK (response says out of scope)
<i>Actual</i>	200 OK (response says out of scope)
<i>Result</i>	Pass

<i>Field</i>	Value
<i>Test Case ID</i>	TC_CHAT_4
<i>Name</i>	delete chat already in database
<i>Endpoint</i>	/api/chat/send_prompt/{chat_id}/
<i>Method</i>	DELETE
<i>Input</i>	{"....." }
<i>Expected</i>	200 OK
<i>Actual</i>	200 OK
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_CHAT_5
<i>Name</i>	delete chat not in database
<i>Endpoint</i>	/api/chat/send_prompt/{chat_id}/
<i>Method</i>	DELETE
<i>Input</i>	{"....." }
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_CHAT_6
<i>Name</i>	history of chats
<i>Endpoint</i>	/api/chat/history/
<i>Method</i>	GET
<i>Input</i>	{"....." }
<i>Expected</i>	200 OK
<i>Actual</i>	200 OK
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_CHAT_7
<i>Name</i>	get chat with id in database
<i>Endpoint</i>	/api/chat/get_chat/{chat_id}/
<i>Method</i>	GET
<i>Input</i>	{"....." }
<i>Expected</i>	200 OK
<i>Actual</i>	200 OK
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_CHAT_8
<i>Name</i>	get chat with id not in database
<i>Endpoint</i>	/api/chat/get_chat/{chat_id}/
<i>Method</i>	GET
<i>Input</i>	{"....." }
<i>Expected</i>	400 Bad Request
<i>Actual</i>	400 Bad Request
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_CHAT_9
<i>Name</i>	Update chat
<i>Endpoint</i>	/api/chat/chat/{chat_id}/
<i>Method</i>	PUT
<i>Input</i>	{"title": "MyChat1"}
<i>Expected</i>	200 Ok
<i>Actual</i>	200 Ok
<i>Result</i>	Pass

5.3.4 Profile Management Test cases

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_PROFILE_01
<i>Name</i>	Verify the user's handle(valid handle)
<i>Endpoint</i>	/api/profile /verify/
<i>Method</i>	GET
<i>Input</i>	{"handle": "handle id"}
<i>Expected</i>	200 Ok (retrieve a problem)
<i>Actual</i>	200 Ok (retrieve a problem)
<i>Result</i>	Pass

<i>Field</i>	Value
<i>Test Case ID</i>	TC_PROFILE_02
<i>Name</i>	Verify the user's handle(valid handle)
<i>Endpoint</i>	/api/profile /verify/
<i>Method</i>	POST
<i>Input</i>	{"handle": "handle id"}
<i>Expected</i>	200 Ok (verified)
<i>Actual</i>	200 Ok (verified)
<i>Result</i>	Pass

<i>Field</i>	Value
<i>Test Case ID</i>	TC_PROFILE_03
<i>Name</i>	Verify the user's handle(invalid handle)
<i>Endpoint</i>	/api/profile /verify/
<i>Method</i>	GET
<i>Input</i>	{"handle": "handle id" }
<i>Expected</i>	400 Bad Request (no problem retrieved)
<i>Actual</i>	400 Bad Request (no problem retrieved)
<i>Result</i>	Pass

<i>Field</i>	Value
<i>Test Case ID</i>	TC_PROFILE_04
<i>Name</i>	Verify the user's handle(Invalid handle)
<i>Endpoint</i>	/api/profile /verify/
<i>Method</i>	POST
<i>Input</i>	{"handle": "handle id"}
<i>Expected</i>	400 Bad Request (not verified)
<i>Actual</i>	400 Bad Request (not verified)
<i>Result</i>	Pass

5.4 Front-End Mobile Test Cases

5.4.1 Chat Test cases

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_CHAT_1
<i>Name</i>	create chat
<i>Screen</i>	Home Screen
<i>Steps</i>	1. Go to “PSA icon” 2. Click “New Chat”
<i>Expected</i>	New chat created successfully
<i>Actual</i>	1. New chat created successfully 2. Chat screen opens with empty conversation
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_CHAT_2
<i>Name</i>	Send prompt in scope
<i>Screen</i>	Chat View
<i>Steps</i>	1. Open an existing\new chat 2. Enter prompt: “What do you know about dynamic programming” 3. Click “Send” icon
<i>Expected</i>	LLM responds with correct explanation of dynamic programming
<i>Actual</i>	1. LLM responds with correct explanation of dynamic programming 2. Answer appears in the chat bubble
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_CHAT_3
<i>Name</i>	Send prompt out scope
<i>Screen</i>	Chat View
<i>Steps</i>	1. Open an existing\new chat 2. Enter prompt: "Who is the first person won the Nobel prize" 3. Click "Send" icon
<i>Expected</i>	LLM responds: "Sorry, this question is out of scope."
<i>Actual</i>	Response message: "Sorry, I can only answer competitive programming questions. this question is out of scope"
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_CHAT_4
<i>Name</i>	Delete chat (existing chat)
<i>Screen</i>	Chat List
<i>Steps</i>	1. Got to an existing chat in the chat list 2. Click "Delete" icon
<i>Expected</i>	Chat is removed from history
<i>Actual</i>	Chat is deleted successfully and disappeared from the list
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_CHAT_5
<i>Name</i>	View chat history
<i>Screen</i>	Chat List
<i>Steps</i>	1. Open Chat List\Dashboard 2. Observe the list of previous chats
<i>Expected</i>	Each chat shows name and time of last open
<i>Actual</i>	1. List rendered with clickable items (previous chats) 2. Each chat shows name and time of last open
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_CHAT_6
<i>Name</i>	View specific chat
<i>Screen</i>	Chat View
<i>Steps</i>	1. Click on a chat from chat history 2. Wait for messages to load
<i>Expected</i>	Full conversation appears
<i>Actual</i>	1. Full conversation appears 2. Messages loaded correctly
<i>Result</i>	Pass

Field	Value
Test Case ID	TC_CHAT_7
Name	Update chat
Screen	Chat View
Steps	1. Click on PSA icon to show all history 2. Choose a specific chat and click on edit icon to edit the title of chat
Expected	Alert message: "Chat title updated successfully"
Actual	Toast notification: "Chat's title updated successfully"
Result	Pass

5.4.2 User Profile Management Test cases

Field	Value
Test Case ID	TC_USER_01
Name	Change password with valid old password
Screen	Change Password Screen
Steps	1. From Settings Open the "Change Password" Screen 2. Enter: - Old Password: ""Maro@412"" • New Password: "Marwan@312" • Confirm New Password: "Marwan@312" 3. Click "Change"
Expected	Success Message shown: "Password changed successfully."
Actual	Toast notification "Password changed successfully" – redirected to Home page
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_USER_02
Name	Change password with invalid old password
Screen	Change Password Screen
Steps	1. From Settings Open the "Change Password" Screen 2. Enter: • Old Password: "Maroo@412" (invalid old password) • New Password: "Marwan@312" • Confirm New Password: "Marwan@312" 3. Click "change"
Expected	Error Message shown: "Invalid Old Password Please Try Again ."
Actual	Toast notification "Invalid Old Password\data"
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_USER_03
Name	Change password with empty password
Screen	Change Password
Steps	1. From Settings Open the "Change Password" Screen 2. Enter: • Old Password: "Maro@412" • New Password: "" • Confirm New Password: "" 3. Click "change"
Expected	Alert or Inline validation message appears: "Password is required"
Actual	Inline validation message appears: "Password is required"
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_USER_04
Name	Change password with too short characters (< 8 characters)
Screen	Change Password
Steps	1. From Settings Open the "Change Password" Screen 2. Enter: • Old Password: "Maro@412" • New Password: "Ma@12" • Confirm New Password: " Ma@12" 3. Click "change"
Expected	Alert or Validation error message appears: "Password must be at least 8 characters"
Actual	Toast notification "Password must be at least 8 characters"
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_USER_05
Name	Change password (all uppercase letters)
Screen	Reset Password
Steps	1. From Settings Open the "Change Password" Screen 2. Enter: • Old Password: "Maro@412" • New Password: "MARO@412" • Confirm New Password: " MARO@412" 3. Click "change"
Expected	Alert or Validation error message appears: "Password must include lowercase letters"
Actual	Toast notification "Password must include lowercase letters"
Result	Pass

Field	Value
Test Case ID	TC_USER_06
Name	Change password (all lowercase letters)
Screen	Change Password
Steps	1. From Settings Open the "Change Password" Screen 2. Enter: • Old Password: "Maro@412" • New Password: "maro@412" • Confirm New Password: " maro@412" 3. Click "change"
Expected	Alert or Validation error message appears: "Password must include uppercase letters"
Actual	Toast notification "Password must include uppercase letters"
Result	Pass

Field	Value
Test Case ID	TC_USER_07
Name	Change password with no symbols
Screen	Change Password
Steps	1. From Settings Open the "Change Password" Screen 2. Enter: • Old Password: "Maro@412" • New Password: "Maroo1412" • Confirm New Password: " Maroo1412" 3. Click "change"
Expected	Alert or Validation error message appears: "Password must include symbols"
Actual	Toast notification "Password must include symbols"
Result	Pass

Field	Value
Test Case ID	TC_ USER_08
Name	Change password with no numbers
Screen	Change Password
Steps	1. From Settings Open the "Change Password" Screen 2. Enter: • Old Password: "Maro@412" • New Password: "Marawan@" • Confirm New Password: " Marawan@" 3. Click "change"
Expected	Alert or Validation error message appears: “Password must include numbers”
Actual	Toast notification “Password must include numbers”
Result	Pass

Field	Value
Test Case ID	TC_ USER_09
Name	Change password with no alphabetic characters
Screen	Change Password
Steps	1. From Settings Open the "Change Password" Screen 2. Enter: • Old Password: "Maro@412" • New Password: "12345678@" • Confirm New Password: " 12345678@" 3. Click "change"
Expected	Alert or Validation error message appears: “Password must include letters”
Actual	Toast notification “Password must include letters”
Result	Pass

<i>Field</i>	Value
<i>Test Case ID</i>	TC_ USER_10
<i>Name</i>	Delete All Chats From History
<i>Screen</i>	Settings Screen
<i>Steps</i>	From Settings Click " Delete All Chats"
<i>Expected</i>	Alert or Validation error message appears: “All chats are deleted successfully”
<i>Actual</i>	Toast notification “All chats are deleted successfully”
<i>Result</i>	Pass

<i>Field</i>	Value
<i>Test Case ID</i>	TC_ USER_11
<i>Name</i>	Logout
<i>Screen</i>	Settings Screen
<i>Steps</i>	From Settings Click " Logout"
<i>Expected</i>	Redirected to Login Screen
<i>Actual</i>	User logged out and redirected to login Screen
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_USER_12
<i>Name</i>	Add Codeforces handle with valid handle
<i>Screen</i>	Profile page
<i>Steps</i>	1. Go to "Profile" Screen 2. Enter • Codeforces Handle “Maro” 3. Click “Add handle” icon
<i>Expected</i>	Success Message: “Handle linked successfully”
<i>Actual</i>	Success Message: “Handle linked successfully” and: 1. Retrieved Problems solved for each level 2. Problems solved for each type of contests
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_USER_13
<i>Name</i>	Add Codeforces handle with invalid handle
<i>Screen</i>	Profile page
<i>Steps</i>	1. Go to "Profile" Screen 2. Enter • Codeforces Handle “Maroooz12” (invalid handle) 3. Click “Add handle” icon
<i>Expected</i>	Error Message: “Handle not found on Codeforces”
<i>Actual</i>	Toast notification “Handle not found on Codeforces” – “Handle not connected”
<i>Result</i>	Pass

Field	Value
Test Case ID	TC_USER_14
Name	Add AtCoder handle with valid handle
Screen	Profile page
Steps	1. Go to "Profile" Screen 2. Enter: • AtCoder Handle “Maro” 3. Click “Add handle” icon
Expected	Success Message: “Handle linked successfully”
Actual	Success Message: “Handle linked successfully” and Retrieved Problems solved for each Tag
Result	Pass

Field	Value
Test Case ID	TC_USER_15
Name	Add AtCoder handle with invalid handle
Screen	Profile page
Steps	1. Go to "Profile" Screen 2. Enter: • AtCoder Handle “Maroooz12” (invalid handle) 3. Click “Add handle” icon
Expected	Error Message: “Handle not found on AtCoder”
Actual	Toast notification “Handle not found on AtCoder” – “Handle not connected”
Result	Pass

Field	Value
Test Case ID	TC_ USER_16
Name	Verifying user's handle (valid handle) Codeforces / Atcoder
Screen	Profile page
Steps	1. Go to "Profile" Screen 2. Enter: • Handle “Marooooz12” (valid handle) 3. Click “Add handle” icon 4. Pop up link appears to solve a random problem with timer(send a compilation error just to make sure the verification)
Expected	Success Message: “Handle verified successfully”
Actual	Toast notification “Handle verified successfully”
Result	Pass

Field	Value
Test Case ID	TC_ USER_17
Name	Verifying user's handle (invalid handle) not in Codeforces / Atcoder
Screen	Profile page
Steps	1. Go to "Profile" Screen 2. Enter: • Handle “Maroooozzz12” (invalid handle) 3. Click “Add handle” icon
Expected	Error Message: “Handle can't be verified”
Actual	No pop up link appeared to solve a problem - Toast notification “Handle can't be verified”
Result	Pass

5.4.3 Authentication Test cases

5.4.3.1 Registration Test cases

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_01
Name	Register with valid data
Screen	Registration Screen
Steps	1. Open the registration Screen 2. Fill the form with: • First Name: "Marwan" • Last Name: "Ahmed" • Username: "maro123" • Email: "maro@example.com" • Password: "StrongPass@123" 3. Click "Register"
Expected	• User is redirected to the login Screen • Success message appears: "Account created successfully"
Actual	Toast notification "Account created successfully"- User is redirected to login Screen
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_02
Name	Register with empty fields
Screen	Registration Screen
Inputs	All fields left blank
Expected	Form shows validation errors under each field
Actual	Inline validation errors under each field - Account not created due to validation errors
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_03
Name	Register with invalid Password (no Symbol)
Screen	Registration Screen
Steps	1. Open the registration Screen 2. Fill the form with: • First Name: "Marwan" • Last Name: "Ahmed" • Username: "maro123" • Email: "maro@example.com" • Password: "StrongPass123" 3. Click "Register"
Expected	Alert or inline validation message appears: “Password must include a special character”
Actual	Toast notification “Password must include a special character”
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_04
Name	Register with invalid Password (no upper letters)
Screen	Registration Screen
Steps	1. Open the registration Screen 2. Fill the form with: • First Name: "Marwan" • Last Name: "Ahmed" • Username: "maro123" • Email: "maro@example.com" • Password: " strongpass123" 3. Click "Register"
Expected	Alert or inline validation message appears: “Password must include upper characters”
Actual	Toast notification “Password must include upper characters”
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_05
Name	Register with invalid Password (no lower letters)
Screen	Registration Screen
Steps	1. Open the registration Screen 2. Fill the form with: • First Name: "Marwan" • Last Name: "Ahmed" • Username: "maro123" • Email: "maro@example.com" • Password: "STRONGPASS123" 3. Click "Register"
Expected	Alert or inline validation message appears: "Password must include lower characters"
Actual	Toast notification "Password must include lower characters"
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_06
Name	Register with invalid Password (no numeric)
Screen	Registration Screen
Steps	1. Open the registration Screen 2. Fill the form with: • First Name: "Marwan" • Last Name: "Ahmed" • Username: "maro123" • Email: "maro@example.com" • Password: " StrongPass@" 3. Click "Register"
Expected	Alert or inline validation message appears: "Password must include at least one number"
Actual	Toast notification "Password must include at least one number"
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_07
Name	Register with invalid Password (short password less than 8 characters)
Screen	Registration Screen
Steps	1. Open the registration Screen 2. Fill the form with: • First Name: "Marwan" • Last Name: "Ahmed" • Username: "maro123" • Email: "maro@example.com" • Password: " S@123" 3. Click "Register"
Expected	Alert or inline validation message appears: “Password must be at least 8 characters”
Actual	Toast notification “Password must be at least 8 characters”
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_08
Name	Register with invalid Password (no characters – only symbols and numbers)
Screen	Registration Screen
Steps	1. Open the registration Screen 2. Fill the form with: • First Name: "Marwan" • Last Name: "Ahmed" • Username: "maro123" • Email: "maro@example.com" • Password: "@123" 3. Click "Register"
Expected	Alert or inline validation message appears: “Password must include letters”
Actual	Toast notification “Password must include letters”
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_09
Name	Register with invalid Password (null)
Screen	Registration Screen
Steps	1. Open the registration Screen 2. Fill the form with: • First Name: "Marwan" • Last Name: "Ahmed" • Username: "maro123" • Email: "maro@example.com" • Password: "" (empty) 3. Click "Register"
Expected	Alert or inline validation message appears: "Password is required"
Actual	Toast notification "Password is required"
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_10
Name	Register with invalid First Name (null)
Screen	Registration Screen
Steps	1. Open the registration Screen 2. Fill the form with: • First Name: "" • Last Name: "Test" • Username: "user1" • Email: "marawan.ah.ab@gmail.com" • Password: "StrongPass@123" 3. Click "Register"
Expected	Alert or inline validation message appears: "First name is required"
Actual	Toast notification\inline validation error message "First name is required"
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_11
Name	Register with invalid First Name (contains numbers)
Screen	Registration Screen
Steps	1. Open the registration Screen 2. Fill the form with: • First Name: "test1" • Last Name: "Test" • Username: "user1" • Email: "marawan.ah.ab@gmail.com" • Password: "StrongPass@123" 3. Click "Register"
Expected	Alert or Inline validation message: “First name must contain only letters”
Actual	Toast notification: “First name must contain only letters”
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_12
Name	Register with invalid First Name (contains symbols)
Screen	Registration Screen
Steps	1. Open the registration Screen 2. Fill the form with: • First Name: "test@" • Last Name: "Test" • Username: "user1" • Email: "marawan.ah.ab@gmail.com" • Password: "StrongPass@123" 3. Click "Register"
Expected	Alert or Inline validation message: “First name must not contain symbols”
Actual	Toast notification: “First name must not contain symbols”
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_13
Name	Register with invalid Last Name (null)
Screen	Registration Screen
Steps	1. Open the registration Screen 2. Fill the form with: • First Name: "User" • Last Name: "" • Username: "user1" • Email: "marawan.ah.ab@gmail.com" • Password: "StrongPass@123" 3. Click "Register"
Expected	Alert or Inline validation message: “Last name is required”
Actual	Toast notification\inline validation error message: “Last name is required”
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_14
Name	Register with invalid Last Name (contains numbers)
Screen	Registration Screen
Steps	1. Open the registration Screen 2. Fill the form with: • First Name: "test" • Last Name: "Test1" • Username: "user1" • Email: "marawan.ah.ab@gmail.com" • Password: "StrongPass@123" 3. Click "Register"
Expected	Alert or Inline validation message: “Last name must contain only letters”
Actual	Toast notification: “Last name must contain only letters”
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_15
Name	Register with invalid Last Name (contains symbols)
Screen	Registration Screen
Steps	1. Open the registration Screen 2. Fill the form with: • First Name: "test" • Last Name: "Test@" • Username: "user1" • Email: "marawan.ah.ab@gmail.com" • Password: "StrongPass@123" 3. Click "Register"
Expected	Alert or Inline validation message: “Last name must contain only letters not symbols”
Actual	Toast notification: “Last name must contain only letters not symbols”
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_16
Name	Register with invalid Username (null)
Screen	Registration Screen
Steps	1. Open the registration Screen 2. Fill the form with: • First Name: "Marwan" • Last Name: "Ahmed" • Username: "" • Email: "maro@example.com" • Password: "StrongPass@123" 3. Click "Register"
Expected	Alert or Inline validation message: “Username is required”
Actual	Toast notification\ Inline validation error message: “Username is required”
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_17
Name	Register with duplicate Username (already registered)
Screen	Registration Screen
Steps	1. Open the registration Screen 2. Fill the form with: • First Name: "Marwan" • Last Name: "Ahmed" • Username: "user1" (already in database) • Email: "newemail@example.com" • Password: "StrongPass@123" 3. Click "Register"
Expected	Alert or Inline validation message: "Username is already taken"
Actual	Toast notification: "Username is already taken"
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_18
Name	Registration – Invalid email format
Screen	Registration Screen
Steps	1. Open the registration Screen 2. Fill the form with: • First Name: "Marwan" • Last Name: "Ahmed" • Username: "maro123" • Email: "maro.example.com" • Password: "StrongPass@123" 3. Click "Register"
Expected	Alert or inline validation message appears: "Enter a valid email address"
Actual	Toast notification "Enter a valid email address"
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_19
Name	Register with invalid Email (null)
Screen	Registration Screen
Steps	1. Open the registration Screen 2. Fill the form with: • First Name: "Marwan" • Last Name: "Ahmed" • Username: "newuser" • Email: "" • Password: "StrongPass@123" 3. Click "Register"
Expected	Alert or Inline validation message: "Email is required"
Actual	Toast notification\ Inline validation error message: "Email is required"
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_20
Name	Register with duplicate Email (already registered)
Screen	Registration Screen
Steps	1. Open the registration Screen 2. Fill the form with: • First Name: "Marwan" • Last Name: "Ahmed" • Username: "uniqueUser" • Email: "marawan.ah.ab@gmail.com" (already in database) • Password: "StrongPass@123" 3. Click "Register"
Expected	Alert or Inline validation message: "Email is already in use"
Actual	Toast notification: "Email is already in use"
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_21
Name	Wrong Confirmation password
Screen	Registration Screen
Steps	1. Open the registration Screen 2. Fill the form with: • First Name: "Marwan" • Last Name: "Ahmed" • Username: "uniqueUser" • Email: "marawan.ah.ab@gmail.com" • Password: "StrongPass@123" • Confirm Password "StrongPass@122" 3. Click "Register"
Expected	Alert or Inline validation message: "Password and confirmation password doesn't match"
Actual	Toast notification: "Password and confirmation password doesn't match"
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_22
Name	Password similar to username or email
Screen	Registration Screen
Steps	1. Open the registration Screen 2. Fill the form with: • First Name: "Marwan" • Last Name: "Ahmed" • Username: "MarawanAhmed" • Email: "marawan.ah.ab@gmail.com" • Password: " MarawanAhmed@123" • Confirm Password "StrongPass@122" 3. Click "Register"
Expected	Alert or Inline validation message: "password is too similar to username or email"
Actual	Toast notification: "password is too similar to username or email"

Result	Pass
---------------	------

5.4.3.2 Login Test cases

Field	Value
Test Case ID	TC_AUTH_23
Name	Login with valid data
Screen	Login Screen
Steps	1. Open the login Screen 2. Enter: • Email: "marawan.ah.ab@example.com" • Password: "StrongPass@123" 3. Click "Login"
Expected	Redirected to home Screen
Actual	User successfully redirected to home Screen
Result	Pass

Field	Value
Test Case ID	TC_AUTH_24
Name	Login with null email
Screen	Login Screen
Steps	1. Open the login Screen 2. Enter: • Email: "" • Password: "StrongPass@123" 3. Click "Login"
Expected	Alert or Validation error message appears: "Email is required"
Actual	Inline validation error message appears "Email is required"
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_25
Name	Login with incorrect email(not exist in database)
Screen	Login Screen
Steps	1. Open the login Screen 2. Enter: • Email: "nonexistent@example.com" • Password: "StrongPass@123" 3. Click "Login"
Expected	Alert or Validation error message appears: “Invalid email or password”
Actual	Toast notification “Invalid email or password”
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_26
Name	Login with empty password
Screen	Login Screen
Steps	1. Open the login Screen 2. Enter: • Email: "marawan.ah.ab@gmail.com" • Password: "" 3. Click "Login"
Expected	Alert or Validation error message appears: “Password is required”
Actual	Inline validation error message appears “Password is required”
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_27
Name	Login with incorrect password
Screen	Login Screen
Steps	1. Open the login Screen 2. Enter: • Email: "marawan.ah.ab@gmail.com" • Password: "IncorrectPass@123" 3. Click "Login"
Expected	Alert or Validation error message appears: "Invalid Email or Password"
Actual	Inline validation error message appears "Invalid Email or Password"
Result	Pass

5.4.3.3 OTP Test cases

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_28
Name	Request OTP with valid email
Screen	Email Verification Screen
Steps	1. Open "Verify Email" page 2. Enter: • Email: "marawan.ah.ab@gmail.com" 3. Click "Send OTP\Check Account"
Expected	Toast notification: "OTP has been sent to your email"
Actual	Confirmation toast notification appears "OTP has been sent to your email"
Result	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_29
<i>Name</i>	Request OTP with empty email
<i>Screen</i>	Email Verification Screen
<i>Steps</i>	1. Open "Verify Email" page 2. Leave email field empty 3. Click "Send OTP\Check Account "
<i>Expected</i>	Alert or Inline validation error message: “Email is required” - OTP not sent
<i>Actual</i>	Inline validation error message: “Email is required” - OTP not sent
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_30
<i>Name</i>	Request OTP with invalid email (missing @)
<i>Screen</i>	Email Verification Screen
<i>Steps</i>	1. Open “Verify Email” page 2. Enter: • Email: "test.gmail.com" 3. Click "Send OTP\Check Account"
<i>Expected</i>	Alert or Validation error message appears: “Enter a valid email address” - OTP not sent
<i>Actual</i>	Toast notification “Enter a valid email address” - OTP not sent
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_31
<i>Name</i>	Request OTP with invalid email (missing domain)
<i>Screen</i>	Email Verification Screen
<i>Steps</i>	1. Open “Verify Email” page 2. Enter: • Email: "test@.com" 3. Click "Send OTP\Check Account"
<i>Expected</i>	Alert or Validation error message appears: “Enter a valid email address” - OTP not sent
<i>Actual</i>	Toast notification “Enter a valid email address” - OTP not sent
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_32
<i>Name</i>	Submit correct OTP
<i>Screen</i>	OTP Verification Screen
<i>Steps</i>	1. Go to OTP input screen 2. Enter: "123456" (correct OTP) 3. Click "Verify\Done"
<i>Expected</i>	Success message: “Email verified successfully”
<i>Actual</i>	Toast notification appears “Email verified successfully”
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_33
<i>Name</i>	Submit incorrect OTP
<i>Screen</i>	OTP Verification Screen
<i>Steps</i>	1. Go to OTP input screen 2. Enter: "654321" (wrong OTP) 3. Click "Verify\Done"
<i>Expected</i>	Error message: "Incorrect OTP"
<i>Actual</i>	Toast notification appears "Incorrect OTP\invalid data"
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_34
<i>Name</i>	Submit OTP after registration (valid Email but not exist)
<i>Screen</i>	OTP Verification Screen
<i>Steps</i>	1. Go to OTP input screen 2. Enter: "" (OTP not sent) 3. Click "Verify\Done"
<i>Expected</i>	OTP not sent as the email doesn't Exist
<i>Actual</i>	OTP not sent as the email doesn't Exist
<i>Result</i>	Pass

5.4.3.4 Check Email test Cases

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_35
<i>Name</i>	Check email with valid registered email
<i>Screen</i>	Check Account Screen
<i>Steps</i>	1. Go to "Forgot\Reset Password" 2. Enter: • Email: "marawan.ah.ab@gmail.com" 3. Click "Check Email"
<i>Expected</i>	Success Message shown: "Email exists\You can reset your password."
<i>Actual</i>	Toast notification "Email verified successfully" – redirected to OTP verification
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_36
<i>Name</i>	Check email with empty input
<i>Screen</i>	Check Account Screen
<i>Steps</i>	1. Go to "Forgot\Reset Password" 2. Leave the email field empty 3. Click "Check Email"
<i>Expected</i>	Alert or Inline validation message appears: "Email is required"
<i>Actual</i>	Inline validation message appears: "Email is required"
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_37
<i>Name</i>	Check email with invalid format (missing @\domain)
<i>Screen</i>	Check Account Screen
<i>Steps</i>	1. Go to "Forgot\Reset Password" 2. Enter: • Email: "test.gmail.com" 3. Click "Check Email"
<i>Expected</i>	Alert or Validation error message appears: “Enter a valid email address”
<i>Actual</i>	Toast notification “Enter a valid email address”
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_38
<i>Name</i>	Check email - not registered
<i>Screen</i>	Check Account
<i>Steps</i>	1. Go to "Forgot\Reset Password" 2. Enter: • Email: "notfound@gmail.com" 3. Click "Check Email"
<i>Expected</i>	Alert or Validation error message appears: “Email not registered\doesn’t exist”
<i>Actual</i>	Toast notification “Email not registered”
<i>Result</i>	Pass

5.4.3.5 Reset Password test cases

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_39
<i>Name</i>	Reset password with valid input
<i>Screen</i>	Reset Password
<i>Steps</i>	4. Open the "Reset Password" screen 5. Enter: • New Password: "Marwan@312" • Confirm New Password: "Marwan@312" 6. Click "Reset"
<i>Expected</i>	Success Message shown: "Password reset successfully."
<i>Actual</i>	Toast notification "Password reset successfully" – redirected to Login Screen
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_40
<i>Name</i>	Reset password with empty password
<i>Screen</i>	Reset Password
<i>Steps</i>	4. Open the "Reset Password" screen 5. Enter: • New Password: "" • Confirm New Password: "" 6. Click "Reset"
<i>Expected</i>	Alert or Inline validation message appears: "Password is required"
<i>Actual</i>	Inline validation message appears: "Password is required"
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_41
Name	Reset password with too short (< 8 characters)
Screen	Reset Password
Steps	7. Open the "Reset Password" screen 8. Enter: • New Password: " Ma@312" • Confirm New Password: " Ma@312" 9. Click "Reset"
Expected	Alert or Validation error message appears: "Password must be at least 8 characters"
Actual	Toast notification "Password must be at least 8 characters"
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_42
Name	Reset password (all uppercase letters)
Screen	Reset Password
Steps	10. Open the "Reset Password" screen 11. Enter: • New Password: " MARWAN@312" • Confirm New Password: " MARWAN@312" 12. Click "Reset"
Expected	Alert or Validation error message appears: "Password must include lowercase letters"
Actual	Toast notification "Password must include lowercase letters"
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_43
Name	Reset password (all lowercase letters)
Screen	Reset Password
Steps	13. Open the "Reset Password" screen 14. Enter: • New Password: " marwan@312" • Confirm New Password: "marwan@312" 15. Click "Reset"
Expected	Alert or Validation error message appears: "Password must include uppercase letters"
Actual	Toast notification "Password must include uppercase letters"
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_44
Name	Reset password with no symbols
Screen	Reset Password
Steps	16. Open the "Reset Password" screen 17. Enter: • New Password: " marwan312" • Confirm New Password: "marwan312" 18. Click "Reset"
Expected	Alert or Validation error message appears: "Password must include symbols"
Actual	Toast notification "Password must include symbols"
Result	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_45
<i>Name</i>	Reset password with no symbols
<i>Screen</i>	Reset Password
<i>Steps</i>	1. Open the "Reset Password" screen 2. Enter: • New Password: " marwan312" • Confirm New Password: "marwan312" 3. Click "Reset"
<i>Expected</i>	Alert or Validation error message appears: “Password must include symbols”
<i>Actual</i>	Toast notification “Password must include symbols”
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_46
<i>Name</i>	Reset password with no numbers
<i>Screen</i>	Reset Password
<i>Steps</i>	19. Open the "Reset Password" screen 20. Enter: • New Password: " Marwan@" • Confirm New Password: Marwan@" 21. Click "Reset"
<i>Expected</i>	Alert or Validation error message appears: “Password must include numbers”
<i>Actual</i>	Toast notification “Password must include numbers”
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_47
Name	Reset password with no alphabetic characters
Screen	Reset Password
Steps	22. Open the "Reset Password" screen 23. Enter: • New Password: " 12345678@" • Confirm New Password: 12345678@" 24. Click "Reset"
Expected	Alert or Validation error message appears: "Password must include letters"
Actual	Toast notification "Password must include letters"
Result	Pass

5.5 Front-end Web Test cases

5.5.1 Chat Test cases

<i>Field</i>	<i>Value</i>
Test Case ID	TC_CHAT_1
Name	create chat
Screen	Home Page
Steps	3. Go to "PSA icon" 4. Click "New Chat"
Expected	New chat created successfully
Actual	3. New chat created successfully 4. Chat screen opens with empty conversation
Result	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_CHAT_2
<i>Name</i>	Send prompt in scope
<i>Screen</i>	Chat View
<i>Steps</i>	4. Open an existing\new chat 5. Enter prompt: “What do you know about dynamic programming” 6. Click “Send” icon
<i>Expected</i>	LLM responds with correct explanation of dynamic programming
<i>Actual</i>	3. LLM responds with correct explanation of dynamic programming 4. Answer appears in the chat bubble
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_CHAT_3
<i>Name</i>	Send prompt out scope
<i>Screen</i>	Chat View
<i>Steps</i>	4. Open an existing\new chat 5. Enter prompt: “Who is the first person won the Nobel prize” 6. Click “Send” icon
<i>Expected</i>	LLM responds: “Sorry, this question is out of scope.”
<i>Actual</i>	Response message: “Sorry, I can only answer competitive programming questions. this question is out of scope”
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_CHAT_4
<i>Name</i>	Delete chat (existing chat)
<i>Screen</i>	Chat List
<i>Steps</i>	3. Got to an existing chat in the chat list 4. Click “Delete” icon
<i>Expected</i>	Chat is removed from history
<i>Actual</i>	Chat is deleted successfully and disappeared from the list
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_CHAT_5
<i>Name</i>	View chat history
<i>Screen</i>	Chat List
<i>Steps</i>	3. Open Chat List\Dashboard 4. Observe the list of previous chats
<i>Expected</i>	Each chat shows name and time of last open
<i>Actual</i>	3. List rendered with clickable items (previous chats) 4. Each chat shows name and time of last open
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_CHAT_6
<i>Name</i>	View specific chat
<i>Screen</i>	Chat View
<i>Steps</i>	3. Click on a chat from chat history 4. Wait for messages to load
<i>Expected</i>	Full conversation appears
<i>Actual</i>	3. Full conversation appears 4. Messages loaded correctly
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_CHAT_7
<i>Name</i>	Update chat
<i>Screen</i>	Chat View
<i>Steps</i>	1. Click on PSA icon to show all history 2. Choose a specific chat and click on edit icon to edit the title of chat
<i>Expected</i>	Alert message: "Chat title updated successfully"
<i>Actual</i>	Toast notification: "Chat's title updated successfully"
<i>Result</i>	Pass

5.5.2 User Profile Management Test cases

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_USER_01
<i>Name</i>	Change password with valid old password
<i>Screen</i>	Change Password Page
<i>Steps</i>	4. From Settings Open the "Change Password" Page 5. Enter: - Old Password: "'Maro@412'" • New Password: "Marwan@312" • Confirm New Password: "Marwan@312" 6. Click "Change"
<i>Expected</i>	Success Message shown: "Password changed successfully."
<i>Actual</i>	Toast notification "Password changed successfully" – redirected to Home page
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_USER_02
<i>Name</i>	Change password with invalid old password
<i>Screen</i>	Change Password Page
<i>Steps</i>	7. From Settings Open the "Change Password" Page 8. Enter: • Old Password: "Maroo@412" (invalid old password) • New Password: "Marwan@312" • Confirm New Password: "Marwan@312" 9. Click "change"
<i>Expected</i>	Error Message shown: "Invalid Old Password Please Try Again ."
<i>Actual</i>	Toast notification "Invalid Old Password\data"
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_ USER_03
<i>Name</i>	Change password with empty password
<i>Screen</i>	Change Password
<i>Steps</i>	4. From Settings Open the "Change Password" Page 5. Enter: • Old Password: "Maro@412" • New Password: "" • Confirm New Password: "" 6. Click "change"
<i>Expected</i>	Alert or Inline validation message appears: “Password is required”
<i>Actual</i>	Inline validation message appears: “Password is required”
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_ USER_04
<i>Name</i>	Change password with too short characters (< 8 characters)
<i>Screen</i>	Change Password
<i>Steps</i>	4. From Settings Open the "Change Password" Page 5. Enter: • Old Password: "Maro@412" • New Password: "Ma@12" • Confirm New Password: " Ma@12" 6. Click "change"
<i>Expected</i>	Alert or Validation error message appears: “Password must be at least 8 characters”
<i>Actual</i>	Toast notification “Password must be at least 8 characters”
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_USER_05
<i>Name</i>	Change password (all uppercase letters)
<i>Screen</i>	Reset Password
<i>Steps</i>	4. From Settings Open the "Change Password" Page 5. Enter: • Old Password: "Maro@412" • New Password: "MARO@412" • Confirm New Password: " MARO@412" 6. Click "change"
<i>Expected</i>	Alert or Validation error message appears: “Password must include lowercase letters”
<i>Actual</i>	Toast notification “Password must include lowercase letters”
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_USER_06
<i>Name</i>	Change password (all lowercase letters)
<i>Screen</i>	Change Password
<i>Steps</i>	4. From Settings Open the "Change Password" Page 5. Enter: • Old Password: "Maro@412" • New Password: "maro@412" • Confirm New Password: " maro@412" 6. Click "change"
<i>Expected</i>	Alert or Validation error message appears: “Password must include uppercase letters”
<i>Actual</i>	Toast notification “Password must include uppercase letters”
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_ USER_07
<i>Name</i>	Change password with no symbols
<i>Screen</i>	Change Password
<i>Steps</i>	4. From Settings Open the "Change Password" Page 5. Enter: • Old Password: "Maro@412" • New Password: "Maroo1412" • Confirm New Password: " Maroo1412" 6. Click "change"
<i>Expected</i>	Alert or Validation error message appears: “Password must include symbols”
<i>Actual</i>	Toast notification “Password must include symbols”
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_ USER_08
<i>Name</i>	Change password with no numbers
<i>Screen</i>	Change Password
<i>Steps</i>	4. From Settings Open the "Change Password" Page 5. Enter: • Old Password: "Maro@412" • New Password: "Marawan@" • Confirm New Password: " Marawan@" 6. Click "change"
<i>Expected</i>	Alert or Validation error message appears: “Password must include numbers”
<i>Actual</i>	Toast notification “Password must include numbers”
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_ USER_09
Name	Change password with no alphabetic characters
Screen	Change Password
Steps	25. From Settings Open the "Change Password" Page 26. Enter: • Old Password: "Maro@412" • New Password: "12345678@" • Confirm New Password: " 12345678@" 27. Click "change"
Expected	Alert or Validation error message appears: “Password must include letters”
Actual	Toast notification “Password must include letters”
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_ USER_10
Name	Delete All Chats From History
Screen	Settings page
Steps	From Settings Click " Delete All Chats"
Expected	Alert or Validation error message appears: “All chats are deleted successfully”
Actual	Toast notification “All chats are deleted successfully”
Result	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_ USER_11
<i>Name</i>	Logout
<i>Screen</i>	Settings page
<i>Steps</i>	From Settings Click " Logout"
<i>Expected</i>	Redirected to Login Page
<i>Actual</i>	User logged out and redirected to login page
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_ USER_12
<i>Name</i>	Add Codeforces handle with valid handle
<i>Screen</i>	Profile page
<i>Steps</i>	4. Go to "Profile" page 5. Enter • Codeforces Handle “Maro” 6. Click “Add handle” icon
<i>Expected</i>	Success Message: “Handle linked successfully”
<i>Actual</i>	Success Message: “Handle linked successfully” and: 3. Retrieved Problems solved for each level 4. Problems solved for each type of contests
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_ USER_13
<i>Name</i>	Add Codeforces handle with invalid handle
<i>Screen</i>	Profile page
<i>Steps</i>	4. Go to "Profile" page 5. Enter • Codeforces Handle “Maroooz12” (invalid handle) 6. Click “Add handle” icon
<i>Expected</i>	Error Message: “Handle not found on Codeforces”
<i>Actual</i>	Toast notification “Handle not found on Codeforces” – “Handle not connected”
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_ USER_14
<i>Name</i>	Add AtCoder handle with valid handle
<i>Screen</i>	Profile page
<i>Steps</i>	4. Go to "Profile" page 5. Enter • AtCoder Handle “Maro” 6. Click “Add handle” icon
<i>Expected</i>	Success Message: “Handle linked successfully”
<i>Actual</i>	Success Message: “Handle linked successfully” and Retrieved Problems solved for each Tag
<i>Result</i>	Pass

Field	Value
Test Case ID	TC_USER_15
Name	Add AtCoder handle with invalid handle
Screen	Profile page
Steps	4. Go to "Profile" page 5. Enter • AtCoder Handle “Maroooz12” (invalid handle) 6. Click “Add handle” icon
Expected	Error Message: “Handle not found on AtCoder”
Actual	Toast notification “Handle not found on AtCoder” – “Handle not connected”
Result	Pass

Field	Value
Test Case ID	TC_USER_16
Name	Verifying user’s handle (valid handle) Codeforces / Atcoder
Screen	Profile page
Steps	5. Go to "Profile" Screen 6. Enter: • Handle “Maroooz12” (valid handle) 7. Click “Add handle” icon 8. Pop up link appears to solve a random problem with timer(send a compilation error just to make sure the verification)
Expected	Success Message: “Handle verified successfully”
Actual	Toast notification “Handle verified successfully”
Result	Pass

Field	Value
Test Case ID	TC_USER_17
Name	Verifying user's handle (invalid handle) not in Codeforces / Atcoder
Screen	Profile page
Steps	4. Go to "Profile" Screen 5. Enter: • Handle “Maroooozzz12” (invalid handle) 6. Click “Add handle” icon
Expected	Error Message: “Handle can’t be verified”
Actual	No pop up link appeared to solve a problem - Toast notification “Handle can’t be verified”
Result	Pass

5.5.3

5.5.4 Authentication Test cases

5.5.4.1 Registration Test cases

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_01
<i>Name</i>	Register with valid data
<i>Screen</i>	Registration Page
<i>Steps</i>	4. Open the registration page 5. Fill the form with: • First Name: "Marwan" • Last Name: "Ahmed" • Username: "maro123" • Email: "maro@example.com" • Password: "StrongPass@123" 6. Click "Register"
<i>Expected</i>	• User is redirected to the login page • Success message appears: "Account created successfully"
<i>Actual</i>	Toast notification "Account created successfully"- User is redirected to login page
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_02
<i>Name</i>	Register with empty fields
<i>Screen</i>	Registration Page
<i>Inputs</i>	All fields left blank
<i>Expected</i>	Form shows validation errors under each field
<i>Actual</i>	Inline validation errors under each field - Account not created due to validation errors
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_03
Name	Register with invalid Password (no Symbol)
Screen	Registration Page
Steps	4. Open the registration page 5. Fill the form with: • First Name: "Marwan" • Last Name: "Ahmed" • Username: "maro123" • Email: "maro@example.com" • Password: "StrongPass123" 6. Click "Register"
Expected	Alert or inline validation message appears: “Password must include a special character”
Actual	Toast notification “Password must include a special character”
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_04
Name	Register with invalid Password (no upper letters)
Screen	Registration Page
Steps	4. Open the registration page 5. Fill the form with: • First Name: "Marwan" • Last Name: "Ahmed" • Username: "maro123" • Email: "maro@example.com" • Password: " strongpass123" 6. Click "Register"
Expected	Alert or inline validation message appears: “Password must include upper characters”
Actual	Toast notification “Password must include upper characters”
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_05
Name	Register with invalid Password (no lower letters)
Screen	Registration Page
Steps	4. Open the registration page 5. Fill the form with: • First Name: "Marwan" • Last Name: "Ahmed" • Username: "maro123" • Email: "maro@example.com" • Password: "STRONGPASS123" 6. Click "Register"
Expected	Alert or inline validation message appears: “Password must include lower characters”
Actual	Toast notification “Password must include lower characters”
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_06
Name	Register with invalid Password (no numeric)
Screen	Registration Page
Steps	4. Open the registration page 5. Fill the form with: • First Name: "Marwan" • Last Name: "Ahmed" • Username: "maro123" • Email: "maro@example.com" • Password: " StrongPass@" 6. Click "Register"
Expected	Alert or inline validation message appears: “Password must include at least one number”
Actual	Toast notification “Password must include at least one number”
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_07
Name	Register with invalid Password (short password less than 8 characters)
Screen	Registration Page
Steps	4. Open the registration page 5. Fill the form with: • First Name: "Marwan" • Last Name: "Ahmed" • Username: "maro123" • Email: "maro@example.com" • Password: " S@123" 6. Click "Register"
Expected	Alert or inline validation message appears: “Password must be at least 8 characters”
Actual	Toast notification “Password must be at least 8 characters”
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_08
Name	Register with invalid Password (no characters – only symbols and numbers)
Screen	Registration Page
Steps	4. Open the registration page 5. Fill the form with: • First Name: "Marwan" • Last Name: "Ahmed" • Username: "maro123" • Email: "maro@example.com" • Password: "@123" 6. Click "Register"
Expected	Alert or inline validation message appears: “Password must include letters”
Actual	Toast notification “Password must include letters”
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_09
Name	Register with invalid Password (null)
Screen	Registration Page
Steps	4. Open the registration page 5. Fill the form with: • First Name: "Marwan" • Last Name: "Ahmed" • Username: "maro123" • Email: "maro@example.com" • Password: "" (empty) 6. Click "Register"
Expected	Alert or inline validation message appears: "Password is required"
Actual	Toast notification "Password is required"
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_10
Name	Register with invalid First Name (null)
Screen	Registration Page
Steps	4. Open the registration page 5. Fill the form with: • First Name: "" • Last Name: "Test" • Username: "user1" • Email: "marawan.ah.ab@gmail.com" • Password: "StrongPass@123" 6. Click "Register"
Expected	Alert or inline validation message appears: "First name is required"
Actual	Toast notification\inline validation error message "First name is required"
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_11
Name	Register with invalid First Name (contains numbers)
Screen	Registration Page
Steps	4. Open the registration page 5. Fill the form with: • First Name: "test1" • Last Name: "Test" • Username: "user1" • Email: "marawan.ah.ab@gmail.com" • Password: "StrongPass@123" 6. Click "Register"
Expected	Alert or Inline validation message: "First name must contain only letters"
Actual	Toast notification: "First name must contain only letters"
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_12
Name	Register with invalid First Name (contains symbols)
Screen	Registration Page
Steps	4. Open the registration page 5. Fill the form with: • First Name: "test@" • Last Name: "Test" • Username: "user1" • Email: "marawan.ah.ab@gmail.com" • Password: "StrongPass@123" 6. Click "Register"
Expected	Alert or Inline validation message: "First name must not contain symbols"
Actual	Toast notification: "First name must not contain symbols"
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_13
Name	Register with invalid Last Name (null)
Screen	Registration Page
Steps	4. Open the registration page 5. Fill the form with: • First Name: "User" • Last Name: "" • Username: "user1" • Email: "marawan.ah.ab@gmail.com" • Password: "StrongPass@123" 6. Click "Register"
Expected	Alert or Inline validation message: "Last name is required"
Actual	Toast notification\inline validation error message: "Last name is required"
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_14
Name	Register with invalid Last Name (contains numbers)
Screen	Registration Page
Steps	4. Open the registration page 5. Fill the form with: • First Name: "test" • Last Name: "Test1" • Username: "user1" • Email: "marawan.ah.ab@gmail.com" • Password: "StrongPass@123" 6. Click "Register"
Expected	Alert or Inline validation message: "Last name must contain only letters"
Actual	Toast notification: "Last name must contain only letters"
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_15
Name	Register with invalid Last Name (contains symbols)
Screen	Registration Page
Steps	4. Open the registration page 5. Fill the form with: • First Name: "test" • Last Name: "Test@" • Username: "user1" • Email: "marawan.ah.ab@gmail.com" • Password: "StrongPass@123" 6. Click "Register"
Expected	Alert or Inline validation message: “Last name must contain only letters not symbols”
Actual	Toast notification: “Last name must contain only letters not symbols”
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_16
Name	Register with invalid Username (null)
Screen	Registration Page
Steps	4. Open the registration page 5. Fill the form with: • First Name: "Marwan" • Last Name: "Ahmed" • Username: "" • Email: "maro@example.com" • Password: "StrongPass@123" 6. Click "Register"
Expected	Alert or Inline validation message: “Username is required”
Actual	Toast notification\ Inline validation error message: “Username is required”
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_17
Name	Register with duplicate Username (already registered)
Screen	Registration Page
Steps	4. Open the registration page 5. Fill the form with: • First Name: "Marwan" • Last Name: "Ahmed" • Username: "user1" (already in database) • Email: "newemail@example.com" • Password: "StrongPass@123" 6. Click "Register"
Expected	Alert or Inline validation message: "Username is already taken"
Actual	Toast notification: "Username is already taken"
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_18
Name	Registration – Invalid email format
Screen	Registration Page
Steps	4. Open the registration page 5. Fill the form with: • First Name: "Marwan" • Last Name: "Ahmed" • Username: "maro123" • Email: "maro.example.com" • Password: "StrongPass@123" 6. Click "Register"
Expected	Alert or inline validation message appears: "Enter a valid email address"
Actual	Toast notification "Enter a valid email address"
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_19
Name	Register with invalid Email (null)
Screen	Registration Page
Steps	4. Open the registration page 5. Fill the form with: • First Name: "Marwan" • Last Name: "Ahmed" • Username: "newuser" • Email: "" • Password: "StrongPass@123" 6. Click "Register"
Expected	Alert or Inline validation message: "Email is required"
Actual	Toast notification\ Inline validation error message: "Email is required"
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_20
Name	Register with duplicate Email (already registered)
Screen	Registration Page
Steps	4. Open the registration page 5. Fill the form with: • First Name: "Marwan" • Last Name: "Ahmed" • Username: "uniqueUser" • Email: "marawan.ah.ab@gmail.com" (already in database) • Password: "StrongPass@123" 6. Click "Register"
Expected	Alert or Inline validation message: "Email is already in use"
Actual	Toast notification: "Email is already in use"
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_21
Name	Wrong Confirmation password
Screen	Registration Page
Steps	4. Open the registration page 5. Fill the form with: • First Name: "Marwan" • Last Name: "Ahmed" • Username: "uniqueUser" • Email: "marawan.ah.ab@gmail.com" • Password: "StrongPass@123" • Confirm Password "StrongPass@122" 6. Click "Register"
Expected	Alert or Inline validation message: "Password and confirmation password doesn't match"
Actual	Toast notification: "Password and confirmation password doesn't match"
Result	Pass

5.5.4.2 Login Test cases

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_22
Name	Login with valid data
Screen	Login Page
Steps	4. Open the login page 5. Enter: • Email: "marawan.ah.ab@example.com" • Password: "StrongPass@123" 6. Click "Login"
Expected	Redirected to home page
Actual	User successfully redirected to home page
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_23
Name	Login with null email
Screen	Login Page
Steps	4. Open the login page 5. Enter: • Email: "" • Password: "StrongPass@123" 6. Click "Login"
Expected	Alert or Validation error message appears: "Email is required"
Actual	Inline validation error message appears "Email is required"
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_24
Name	Login with incorrect email(not exist in database)
Screen	Login Page
Steps	4. Open the login page 5. Enter: • Email: "nonexistent@example.com" • Password: "StrongPass@123" 6. Click "Login"
Expected	Alert or Validation error message appears: "Invalid email or password"
Actual	Toast notification "Invalid email or password"
Result	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_25
<i>Name</i>	Login with empty password
<i>Screen</i>	Login Page
<i>Steps</i>	4. Open the login page 5. Enter: • Email: "marawan.ah.ab@gmail.com" • Password: "" 6. Click "Login"
<i>Expected</i>	Alert or Validation error message appears: "Password is required"
<i>Actual</i>	Inline validation error message appears "Password is required"
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_26
<i>Name</i>	Login with incorrect password
<i>Screen</i>	Login Page
<i>Steps</i>	4. Open the login page 5. Enter: • Email: "marawan.ah.ab@gmail.com" • Password: "IncorrectPass@123" 6. Click "Login"
<i>Expected</i>	Alert or Validation error message appears: "Invalid Email or Password"
<i>Actual</i>	Inline validation error message appears "Invalid Email or Password"
<i>Result</i>	Pass

5.5.4.3 OTP Test cases

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_27
<i>Name</i>	Request OTP with valid email
<i>Screen</i>	Email Verification
<i>Steps</i>	5. Open "Verify Email" page 6. Enter: • Email: "marawan.ah.ab@gmail.com" 7. Click "Send OTP\Check Account"
<i>Expected</i>	Toast notification: "OTP has been sent to your email"
<i>Actual</i>	Confirmation toast notification appears "OTP has been sent to your email"
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_28
<i>Name</i>	Request OTP with empty email
<i>Screen</i>	Email Verification
<i>Steps</i>	4. Open "Verify Email" page 5. Leave email field empty 6. Click "Send OTP\Check Account "
<i>Expected</i>	Alert or Inline validation error message: "Email is required" - OTP not sent
<i>Actual</i>	Inline validation error message: "Email is required" - OTP not sent
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_29
Name	Request OTP with invalid email (missing @)
Screen	Email Verification
Steps	4. Open “Verify Email” page 5. Enter: • Email: "test.gmail.com" 6. Click "Send OTP\Check Account"
Expected	Alert or Validation error message appears: “Enter a valid email address” - OTP not sent
Actual	Toast notification “Enter a valid email address” - OTP not sent
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_30
Name	Request OTP with invalid email (missing domain)
Screen	Email Verification
Steps	4. Open “Verify Email” page 5. Enter: • Email: "test@.com" 6. Click "Send OTP\Check Account"
Expected	Alert or Validation error message appears: “Enter a valid email address” - OTP not sent
Actual	Toast notification “Enter a valid email address” - OTP not sent
Result	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_31
<i>Name</i>	Submit correct OTP
<i>Screen</i>	OTP Verification
<i>Steps</i>	4. Go to OTP input screen 5. Enter: "123456" (correct OTP) 6. Click "Verify\Done"
<i>Expected</i>	Success message: "Email verified successfully"
<i>Actual</i>	Toast notification appears "Email verified successfully"
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_32
<i>Name</i>	Submit incorrect OTP
<i>Screen</i>	OTP Verification
<i>Steps</i>	4. Go to OTP input screen 5. Enter: "654321" (wrong OTP) 6. Click "Verify\Done"
<i>Expected</i>	Error message: "Incorrect OTP"
<i>Actual</i>	Toast notification appears "Incorrect OTP\invalid data"
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_33
Name	Submit OTP after registration (valid Email but not exist)
Screen	OTP Verification
Steps	4. Go to OTP input screen 5. Enter: "" (OTP not sent) 6. Click "Verify\Done"
Expected	OTP not sent as the email doesn't Exist
Actual	OTP not sent as the email doesn't Exist
Result	Pass

5.5.4.4 Check Email test Cases

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_34
Name	Check email with valid registered email
Screen	Check Account
Steps	4. Go to "Forgot\Reset Password" 5. Enter: • Email: "marawan.ah.ab@gmail.com" 6. Click "Check Email"
Expected	Success Message shown: "Email exists\You can reset your password."
Actual	Toast notification "Email verified successfully" – redirected to OTP verification
Result	Pass

<i>Field</i>	Value
<i>Test Case ID</i>	TC_AUTH_35
<i>Name</i>	Check email with empty input
<i>Screen</i>	Check Account
<i>Steps</i>	4. Go to "Forgot\Reset Password" 5. Leave the email field empty 6. Click "Check Email"
<i>Expected</i>	Alert or Inline validation message appears: "Email is required"
<i>Actual</i>	Inline validation message appears: "Email is required"
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_36
Name	Check email with invalid format (missing @\domain)
Screen	Check Account
Steps	4. Go to "Forgot\Reset Password" 5. Enter: • Email: "test.gmail.com" 6. Click "Check Email"
Expected	Alert or Validation error message appears: "Enter a valid email address"
Actual	Toast notification "Enter a valid email address"
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_37
Name	Check email - not registered
Screen	Check Account
Steps	4. Go to "Forgot\Reset Password" 5. Enter: • Email: "notfound@gmail.com" 6. Click "Check Email"
Expected	Alert or Validation error message appears: "Email not registered\doesn't exist"
Actual	Toast notification "Email not registered"
Result	Pass

5.5.4.5 Reset Password test cases

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_38
<i>Name</i>	Reset password with valid input
<i>Screen</i>	Reset Password
<i>Steps</i>	10. Open the "Reset Password" Page 11. Enter: • New Password: "Marwan@312" • Confirm New Password: "Marwan@312" 12. Click "Reset"
<i>Expected</i>	Success Message shown: "Password reset successfully."
<i>Actual</i>	Toast notification "Password reset successfully" – redirected to Login Page
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_39
<i>Name</i>	Reset password with empty password
<i>Screen</i>	Reset Password
<i>Steps</i>	28. Open the "Reset Password" Page 29. Enter: • New Password: "" • Confirm New Password: "" 30. Click "Reset"
<i>Expected</i>	Alert or Inline validation message appears: "Password is required"
<i>Actual</i>	Inline validation message appears: "Password is required"
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_40
Name	Reset password with too short (< 8 characters)
Screen	Reset Password
Steps	31. Open the "Reset Password" Page 32. Enter: • New Password: " Ma@312" • Confirm New Password: " Ma@312" 33. Click "Reset"
Expected	Alert or Validation error message appears: "Password must be at least 8 characters"
Actual	Toast notification "Password must be at least 8 characters"
Result	Pass

<i>Field</i>	<i>Value</i>
Test Case ID	TC_AUTH_41
Name	Reset password (all uppercase letters)
Screen	Reset Password
Steps	34. Open the "Reset Password" Page 35. Enter: • New Password: " MARWAN@312" • Confirm New Password: " MARWAN@312" 36. Click "Reset"
Expected	Alert or Validation error message appears: "Password must include lowercase letters"
Actual	Toast notification "Password must include lowercase letters"
Result	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_42
<i>Name</i>	Reset password (all lowercase letters)
<i>Screen</i>	Reset Password
<i>Steps</i>	37. Open the "Reset Password" Page 38. Enter: • New Password: " marwan@312" • Confirm New Password: "marwan@312" 39. Click "Reset"
<i>Expected</i>	Alert or Validation error message appears: "Password must include uppercase letters"
<i>Actual</i>	Toast notification "Password must include uppercase letters"
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_43
<i>Name</i>	Reset password with no symbols
<i>Screen</i>	Reset Password
<i>Steps</i>	40. Open the "Reset Password" Page 41. Enter: • New Password: " marwan312" • Confirm New Password: "marwan312" 42. Click "Reset"
<i>Expected</i>	Alert or Validation error message appears: "Password must include symbols"
<i>Actual</i>	Toast notification "Password must include symbols"
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_44
<i>Name</i>	Reset password with no symbols
<i>Screen</i>	Reset Password
<i>Steps</i>	4. Open the "Reset Password" Page 5. Enter: • New Password: " marwan312" • Confirm New Password: "marwan312" 6. Click "Reset"
<i>Expected</i>	Alert or Validation error message appears: “Password must include symbols”
<i>Actual</i>	Toast notification “Password must include symbols”
<i>Result</i>	Pass

<i>Field</i>	<i>Value</i>
<i>Test Case ID</i>	TC_AUTH_45
<i>Name</i>	Reset password with no numbers
<i>Screen</i>	Reset Password
<i>Steps</i>	43. Open the "Reset Password" Page 44. Enter: • New Password: " Marwan@" • Confirm New Password: Marwan@" 45. Click "Reset"
<i>Expected</i>	Alert or Validation error message appears: “Password must include numbers”
<i>Actual</i>	Toast notification “Password must include numbers”
<i>Result</i>	Pass

<i>Field</i>	Value
<i>Test Case ID</i>	TC_AUTH_46
<i>Name</i>	Reset password with no alphabetic characters
<i>Screen</i>	Reset Password
<i>Steps</i>	46. Open the "Reset Password" Page 47. Enter: • New Password: " 12345678@" • Confirm New Password: 12345678@" 48. Click "Reset"
<i>Expected</i>	Alert or Validation error message appears: "Password must include letters"
<i>Actual</i>	Toast notification "Password must include letters"
<i>Result</i>	Pass

6 Chapter (6) Conclusions and Future Work

6.1 Conclusions

This project addressed a clear and recurring challenge among computer science students and competitive programmers: the difficulty in developing and improving problem-solving skills efficiently and independently. From the outset, our objective was to create a system that not only assists users in solving coding problems but also guides their learning journey through personalized roadmaps and real-time feedback.

The Problem Solving Assistant (PSA) successfully fulfills the core needs of our target users. Through a ChatBot interface, the system provides solutions to user-submitted problems, debugs user-written code, and recommends learning paths based on their current skill level. The integration of NLP models (ChatGPT APIs), backend support via Django and PostgreSQL, and a frontend built with Flutter and React ensures the system is scalable, accessible, and user-friendly.

Our testing phase validated the core functionalities of the system, including account registration, roadmap generation, code debugging, and problem solution generation. The results proved that the PSA is capable of responding to user queries efficiently and providing accurate and reliable feedback.

Moreover, our use of real-world data from surveys and problem-solving websites helped tailor the platform to actual user pain points, making it not just a theoretical solution but a practical and impactful tool for real users.

6.2 Future Work

In future iterations of the project, several enhancements can be implemented to improve functionality and user experience. One proposed improvement is creating an intelligent agent capable of generating a meaningful chat name based on the first prompt. Another goal is to develop a system that evaluates and assigns a user rating based on their interaction with the chat, providing insights into engagement and usage quality.

Additionally, expanding the model's input capabilities to include image formats will enable more versatile interactions. Incorporating a voice-to-text agent will also allow users to send messages using voice commands, enhancing accessibility. Moreover, enabling the chat to save specific data or commands for reuse in other sessions would improve continuity across conversations. Establishing integrations between the Personal Smart Assistant (PSA) and cloud services like Google Drive will further extend the assistant's utility. Finally, making chat sessions shareable and accessible to other PSA users will foster collaboration and broaden the system's usability.

7 References

Courses IBM Data Science Professional Certificate

<https://www.coursera.org/professional-certificates/ibm-data-science>

Flutter Udemmy

<https://drive.google.com/drive/folders/13qpp1fcqrL6g8fhVjBuhwoK9mUxgkMc7?fbclid=IwAR03OttSkApBxbmbET-7YzO4a9NHyoYAqO0Qg40VnuBGPCssenfCUjyr9-8>

<https://www.udemy.com/course/fluttercourse/>

ChatGPT API

<https://openai.com/api/>

Django

<https://www.coursera.org/learn/developing-applications-with-sql-databases-and-django>

Hugging Face DeepSeek R1

<https://huggingface.co/deepseek-ai/DeepSeek-R1>

USACO Guide

[USACO Guide](#)

CP-Algorithm

[Main Page - Algorithms for Competitive Programming](#)

Codeforces

[Codeforces](#)

Atcoder

[AtCoder](#)

Retrieval Augmented Generation

<https://arxiv.org/abs/2005.11401>

Catch Augmented Generation (CAG)

<https://arxiv.org/html/2412.15605v1>

Comparison between Fine tuning and RAG

<https://arxiv.org/html/2403.01432v3>